

Assignment 5

DATA 260

Name: Kazi Samina Maraj Mumu

SJSU ID: 018324098

Email: Kazisaminamaraj.mumu@sjsu.edu

Date: 10/03/2026

GitHub repository: <https://github.com/SaminaMaraj/data260-4098>

Personal Configuration and Project Information

Item	Value
Domain	Municipal transit incidents
Repository	github.com/SaminaMaraj/data260-4098
Code and results commit	62d2947104ac6641be65d184f189612ef37c8be7
hw5 tag	Later commit; the only changes after 62d2947 are report.pdf and verification.json
Local model	qwen3:8b via Ollama
Hardware	Apple MacBook Pro, M5 chip, 24 GB RAM
Python	3.12.10
Database	MySQL 8.4 (Docker), schema s4098_rel, port 3308

SID4	PORT_BASE	PREFIX	SEED	VERIFY_SEED	DOMAIN_ID
4098	8498	s4098	4098	264098	2

All code, data and experiment results are in commit 62d2947104ac6641be65d184f189612ef37c8be7. The final hw5 tag points to a later commit. The only files changed after 62d2947 are this report (report.pdf) and verification.json. No application or experiment code changed after 62d2947.

For this homework I continued the same municipal transit incident project from HW1 to HW4. I did not make a separate copy of the code. I extended the existing FastAPI + MySQL backend and the React frontend, and I added the new HW5 files under code/mcp/ and tests/. All required evidence files are in reports/hw05/.

Part 1 - Database, API and Redux Client

I. Database

My primary entity is the transit incident. For the second, related entity I added a Route table, because every incident already belonged to a transit line. Before this change, the route was only free text (route_line), so now each incident points to a real Route record with a foreign key.

- **routes:** id (primary key), route_name (primary text), operator (secondary text), route_code (unique, format RT-###), created_at, updated_at.
- **incidents:** id (primary key) and incident_title (primary field), which already existed from HW4, plus the new columns incident_code (unique, format INC-#####), passengers_affected (integer, default 0), route_id (foreign key to routes.id, ON DELETE RESTRICT), created_at and updated_at.

Code - src/hw4_models.py (Route model and the incident foreign key)

```
class Route(Base):
    __tablename__ = "routes"

    id: Mapped[int] = mapped_column(
        Integer,
        primary_key=True,
        autoincrement=True,
    )
    route_name: Mapped[str] = mapped_column(
        String(100),
        nullable=False,
    )
    operator: Mapped[str] = mapped_column(
        String(100),
        nullable=False,
    )
    route_code: Mapped[str] = mapped_column(
        String(20),
        unique=True,
        nullable=False,
    )
    ...

# IncidentRecord: the new foreign key to routes
route_id: Mapped[int] = mapped_column(
    ForeignKey("routes.id", ondelete="RESTRICT"),
    nullable=False,
)
```

I already had 5,000 seeded incidents from HW4, so I wrote a separate migration script (code/migrate_hw05.py) instead of recreating the tables. Before running it, I backed up every table. The script created one route for each distinct route_line, filled route_id for every incident, and generated a unique incident_code. After the migration I checked the counts in MySQL:

```
incidents, with route, with code: (5000, 5000, 5000)
routes: 5
```

This means all 5,000 incidents were kept, every incident has a route, and every incident code is unique. I kept the old route_line column so my HW4 code and the N+1 benchmark still work.

For deletes, the route foreign key uses RESTRICT, and the API also checks first, so a route that still has incidents cannot be deleted (409 Conflict). Passwords are not stored in plain text: the HW4 login stores PBKDF2-SHA256 hashes, and I also changed the older HW3 login in src/auth.py to use a stored hash instead of a plain-text password.

```

Kazi Samina Maraj Mumu
mysql: [Warning] Using a password on the command line interface can be insecure.

+-----+
| Tables_in_s4098_rel |
+-----+
| incident_related_data |
| incidents              |
| routes                 |
| sessions               |
| users                  |
+-----+

+-----+
| id | route_name | operator | route_code | created_at | updated_at |
+-----+
| 1 | VTA Green Line | VTA | RT-001 | 2026-10-02 19:14:54 | 2026-10-02 19:14:54 |
| 2 | VTA Orange Line | VTA | RT-002 | 2026-10-02 19:14:54 | 2026-10-02 19:14:54 |
| 3 | VTA Route 22 | VTA | RT-003 | 2026-10-02 19:14:54 | 2026-10-02 19:14:54 |
| 4 | VTA Route 522 | VTA | RT-004 | 2026-10-02 19:14:54 | 2026-10-02 19:14:54 |
| 5 | VTA Route 55 | VTA | RT-005 | 2026-10-02 19:14:54 | 2026-10-02 19:14:54 |
+-----+

+-----+
| id | incident_code | route_id | passengers_affected | category | incident_title |
+-----+
| 2 | INC-000002 | 1 | 0 | infrastructure-issue | Transit Infrastructure Issue Incident 1 |
| 3 | INC-000003 | 5 | 0 | delay | Transit Delay Incident 2 |
| 4 | INC-000004 | 3 | 0 | infrastructure-issue | Transit Infrastructure Issue Incident 3 |
| 5 | INC-000005 | 2 | 0 | infrastructure-issue | Transit Infrastructure Issue Incident 4 |
| 6 | INC-000006 | 3 | 0 | infrastructure-issue | Transit Infrastructure Issue Incident 5 |
| 7 | INC-000007 | 4 | 0 | service-suspension | Transit Service Suspension Incident 6 |
| 8 | INC-000008 | 2 | 0 | delay | Transit Delay Incident 7 |
| 9 | INC-000009 | 5 | 0 | delay | Transit Delay Incident 8 |
| 10 | INC-000010 | 1 | 0 | service-suspension | Transit Service Suspension Incident 9 |
| 11 | INC-000011 | 4 | 0 | infrastructure-issue | Transit Infrastructure Issue Incident 10 |
+-----+

What's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug data260-4098-mysql
Learn more at https://docs.docker.com/go/debug-cli/
(base) samina@Kazis-MacBook-Pro-2 data260-4098 %

```

Figure 1. MySQL database after the migration: the new routes table (RT-001 to RT-005) and incidents with incident_code and route_id.

II. API Implementation and Postman Tests

I added CRUD endpoints for both entities to the existing /api/hw4 router, plus the relationship endpoint GET /routes/{id}/incidents. All endpoints use Pydantic schemas and require the login session cookie. The route_code and incident_code formats are validated with regular expressions. I tested everything in Postman using a temporary test route and test incident, so my seeded data was not changed, and I deleted both test records at the end.

Method	Endpoint	Purpose
POST / GET / GET / PUT / DELETE	/api/hw4/routes, /routes?skip&limit, /routes/{id}	Route CRUD (list is paginated)
POST / GET / GET / PUT / DELETE	/api/hw4/incidents, /incidents?skip&limit, /incidents/{id}	Incident CRUD (list is paginated)
GET	/api/hw4/routes/{id}/incidents	All incidents for one route

Code - src/hw4_api.py, delete_route (blocks deleting a route that still has incidents)

```

route = db.get(Route, route_id)
if route is None:
    raise HTTPException(status_code=404, detail="Route not found")

incident_count = db.scalar(
    select(func.count())
    .select_from(IncidentRecord)

```

```

        .where(IncidentRecord.route_id == route_id)
    )
    if incident_count:
        raise HTTPException(
            status_code=409,
            detail="Cannot delete a route that still has incidents",
        )

db.delete(route)
db.commit()
return {"message": "Route deleted", "id": route_id}

```

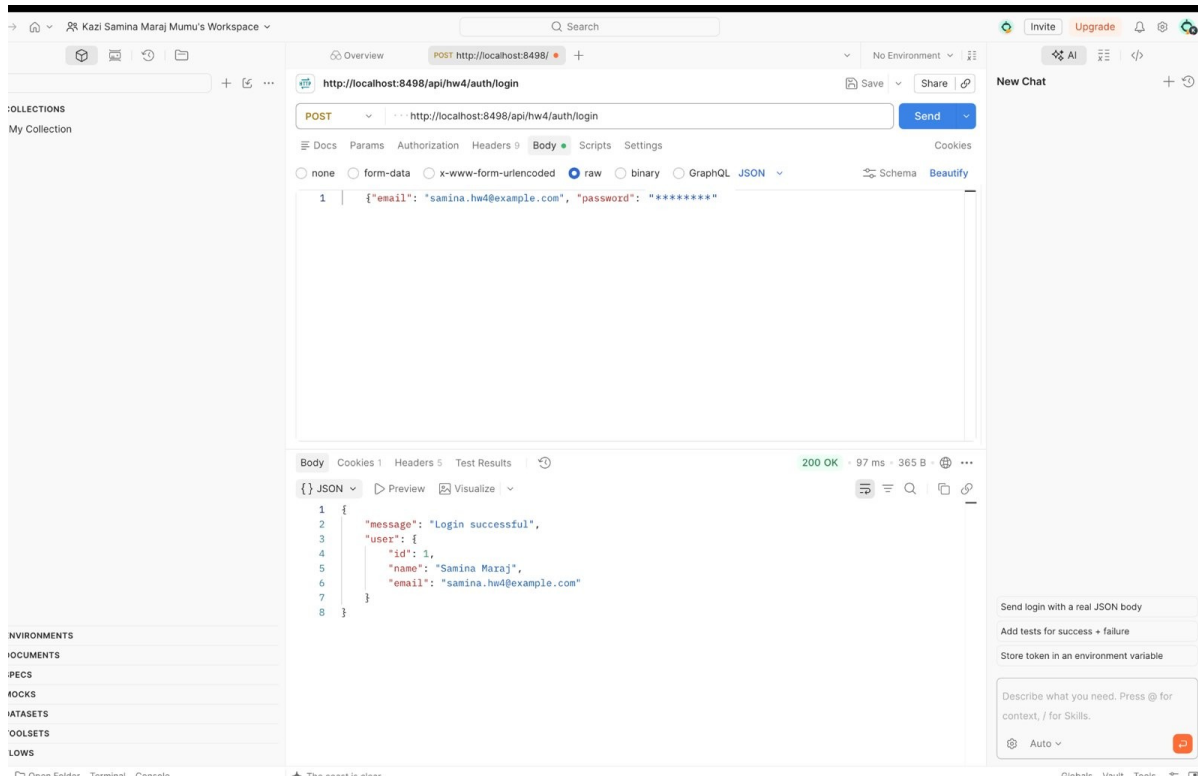


Figure 2. Login in Postman (POST /auth/login) returns 200 and sets the session cookie. Password hidden.

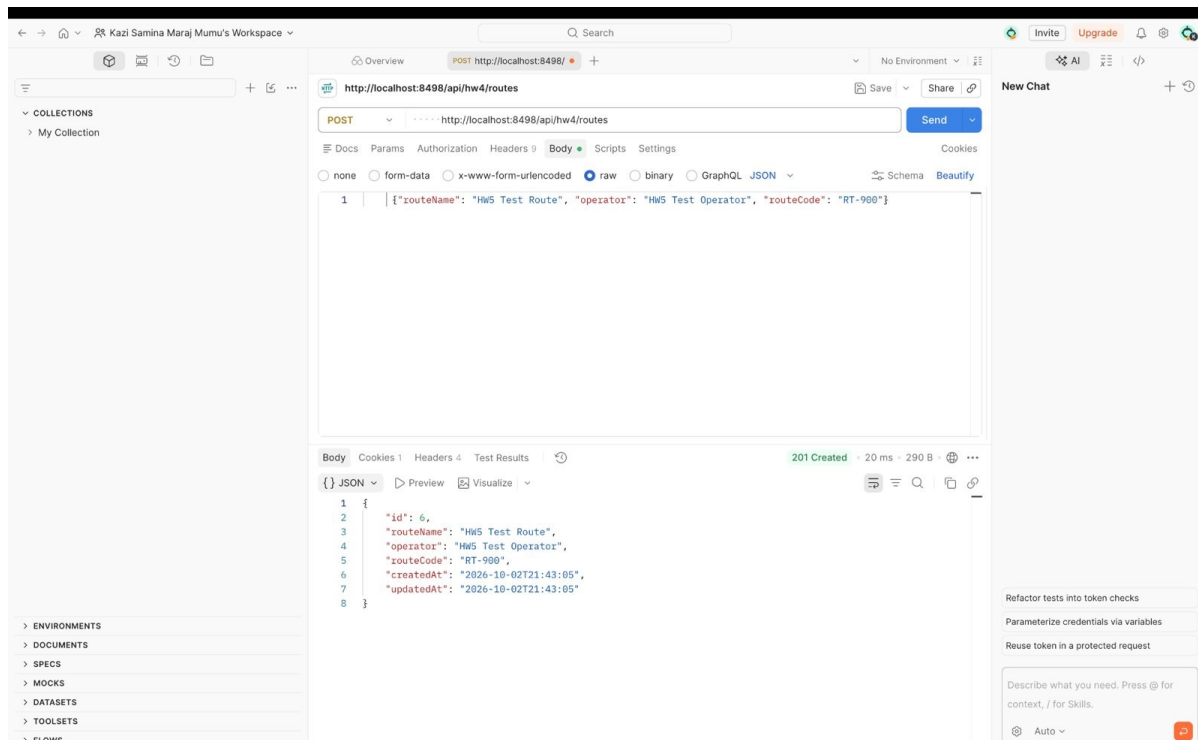


Figure 3. Create a test route (POST /routes) returns 201 Created with id 6 and code RT-900.

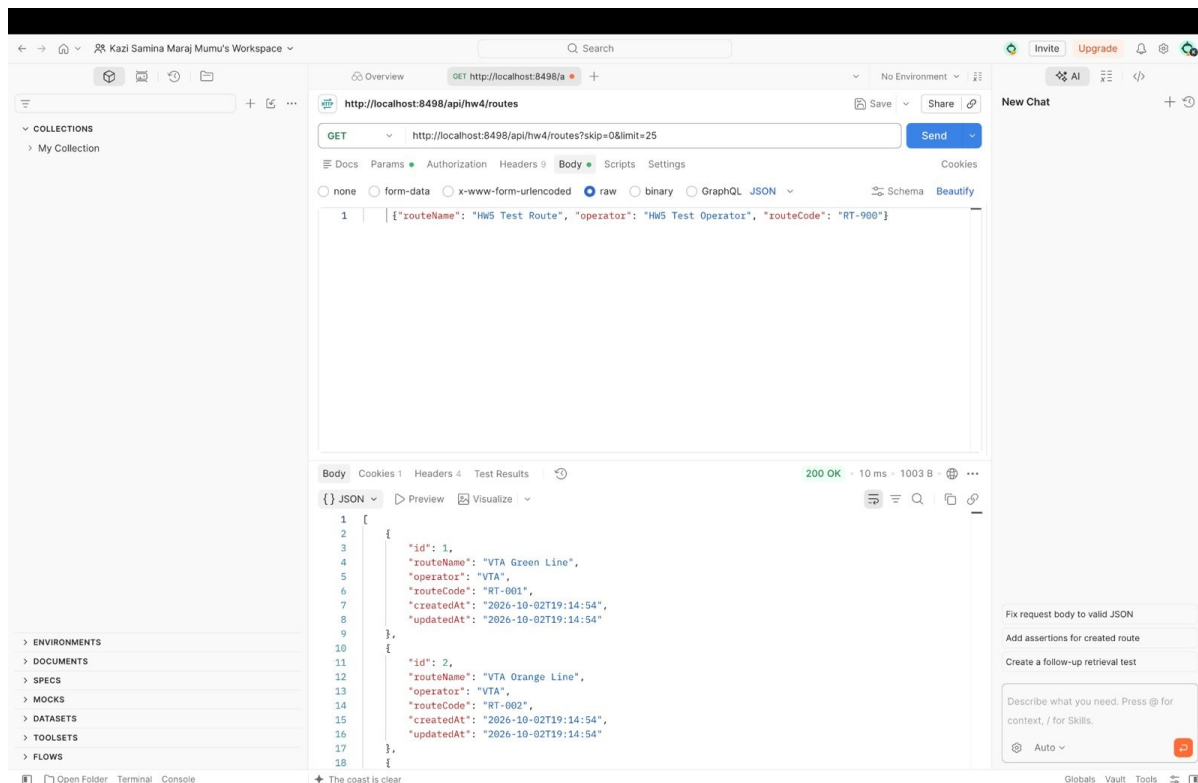


Figure 4. List routes with pagination (GET /routes?skip=0&limit=25) returns 200.

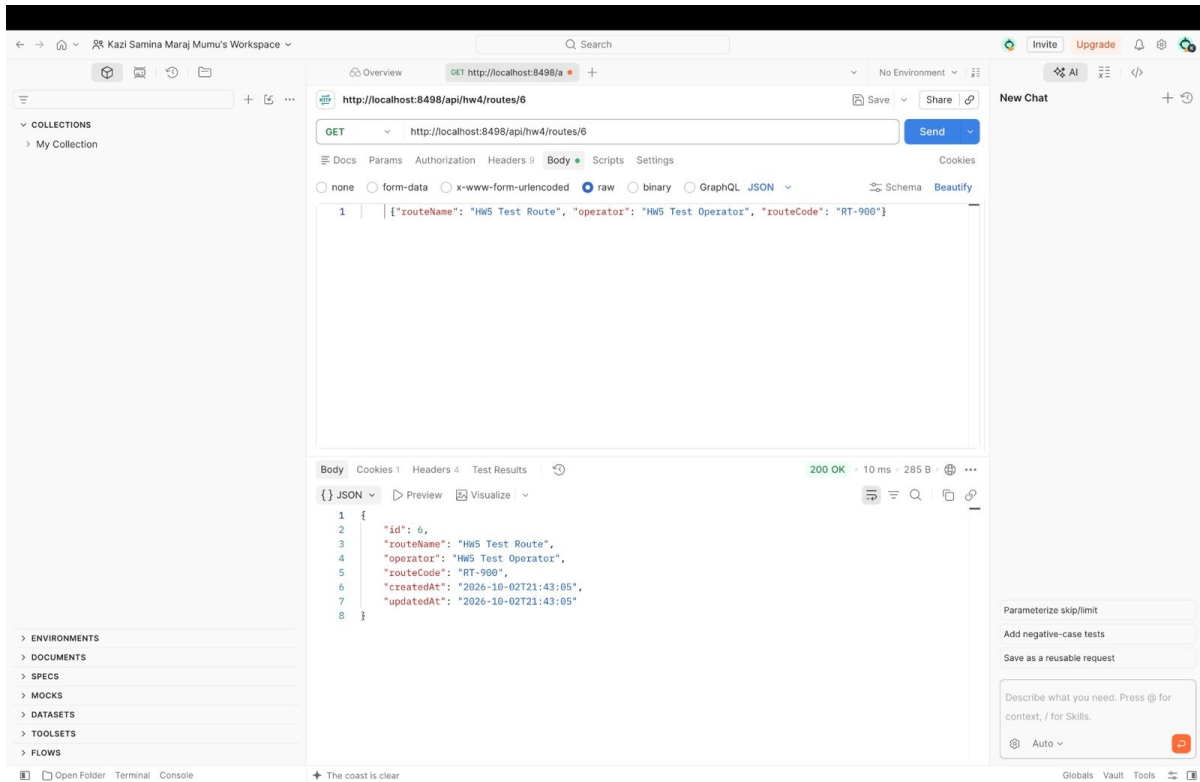


Figure 5. Get one route (GET /routes/6) returns 200.

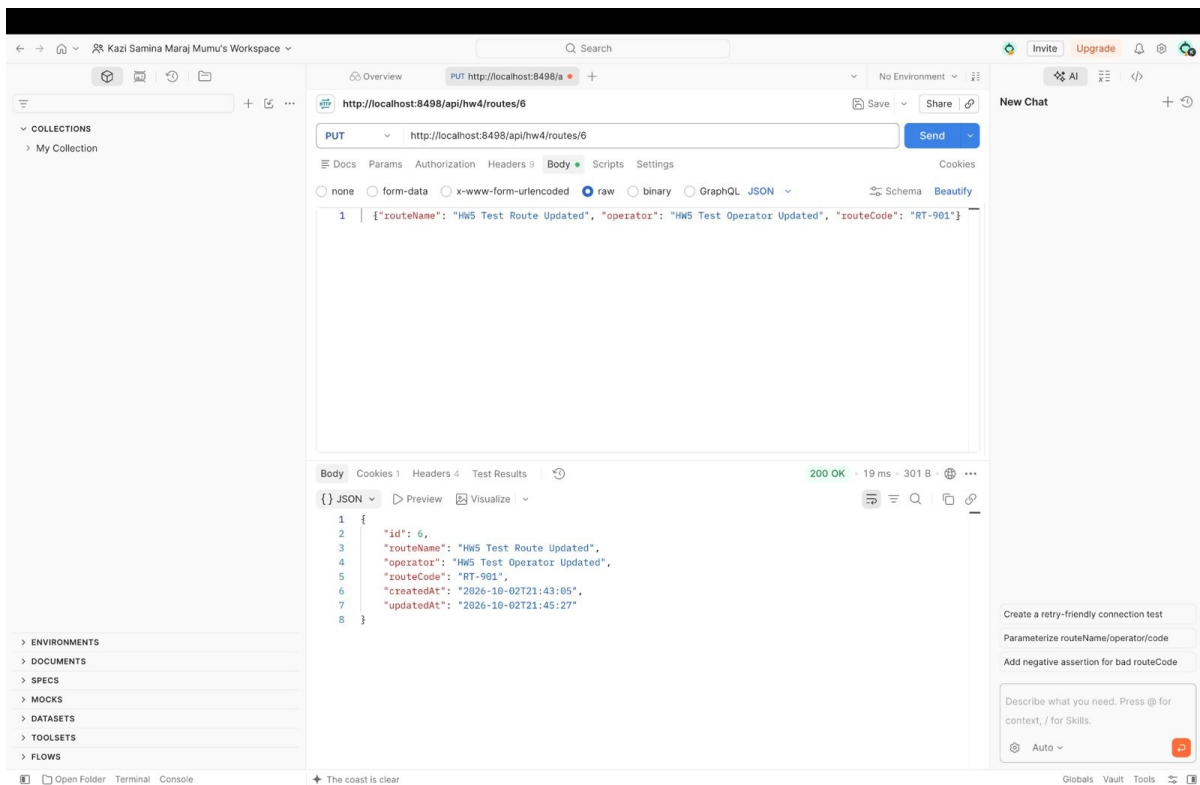


Figure 6. Update the route (PUT /routes/6): new name, operator and code RT-901, 200 OK.

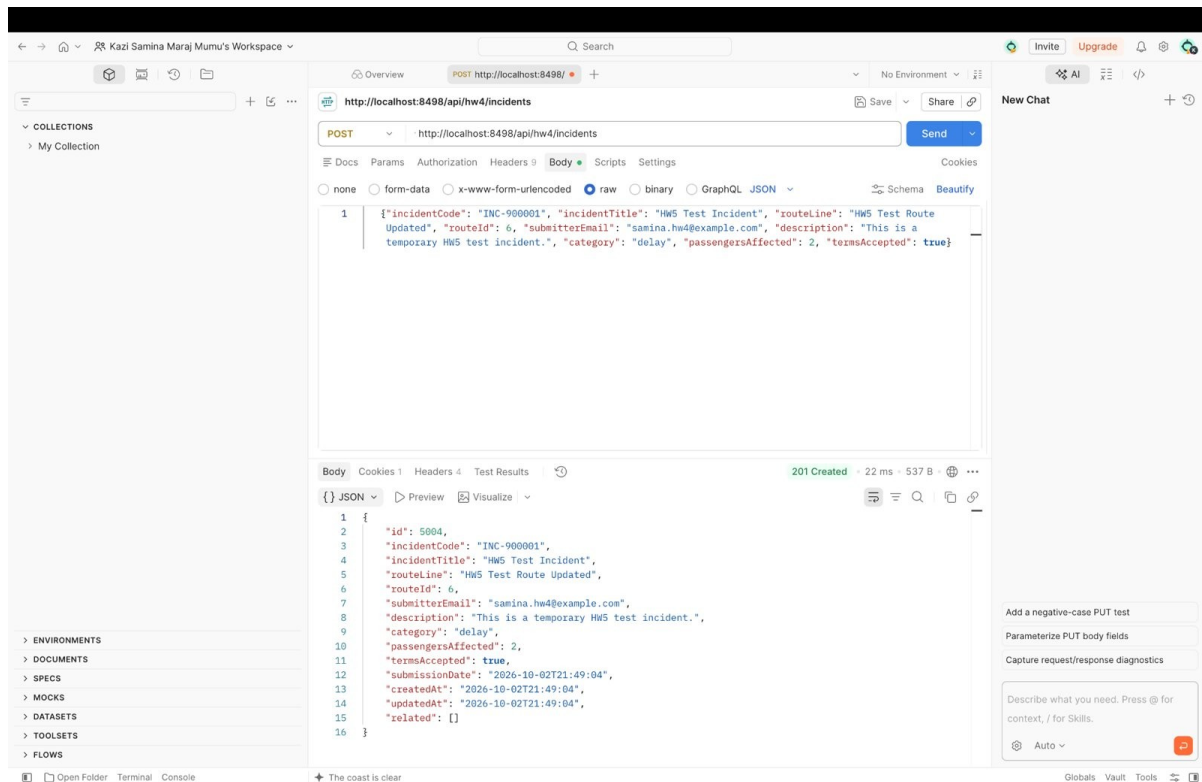


Figure 7. Create a test incident on route 6 (POST /incidents) returns 201 Created with id 5004.

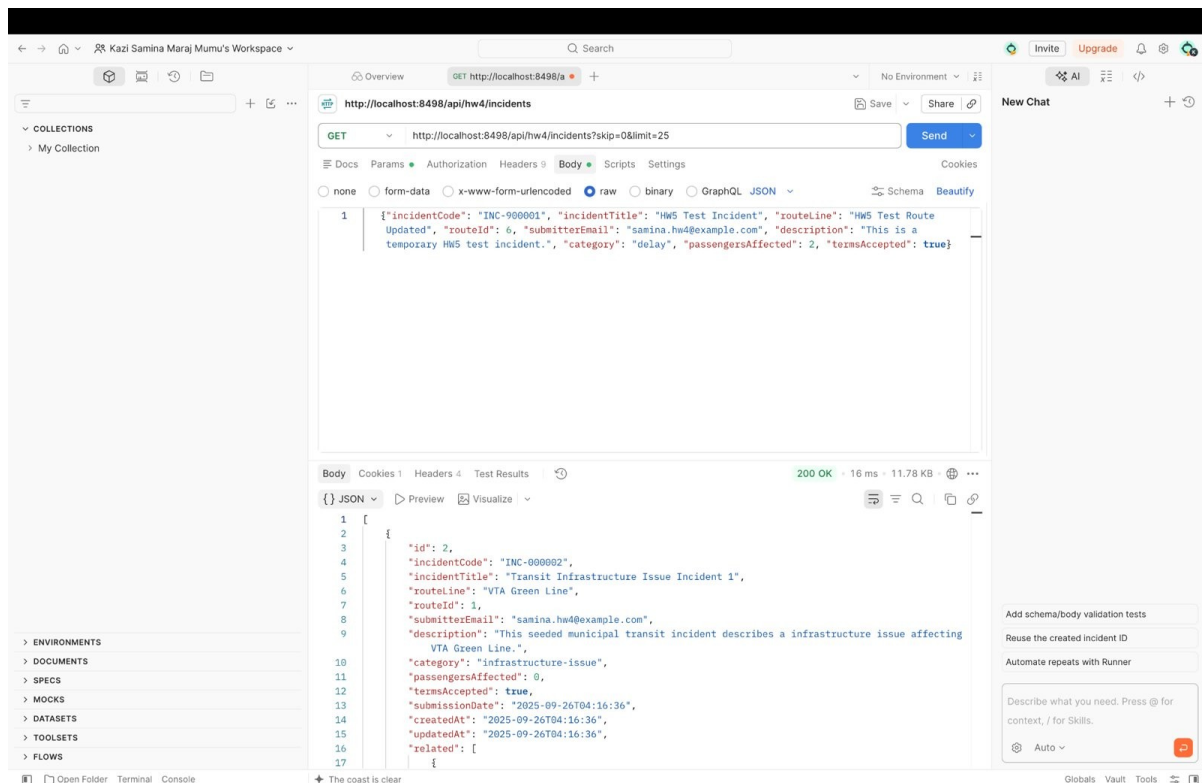


Figure 8. List incidents with pagination (GET /incidents?skip=0&limit=25) returns 200.

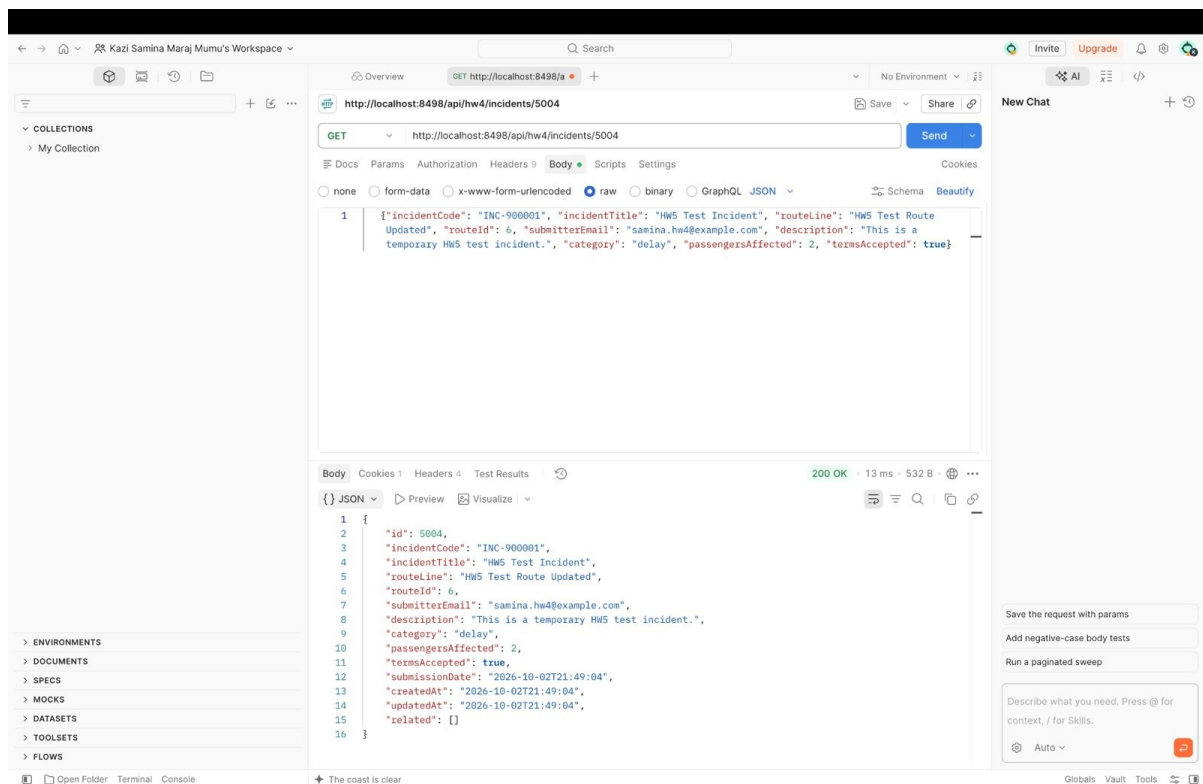


Figure 9. Get one incident (GET /incidents/5004) returns 200.

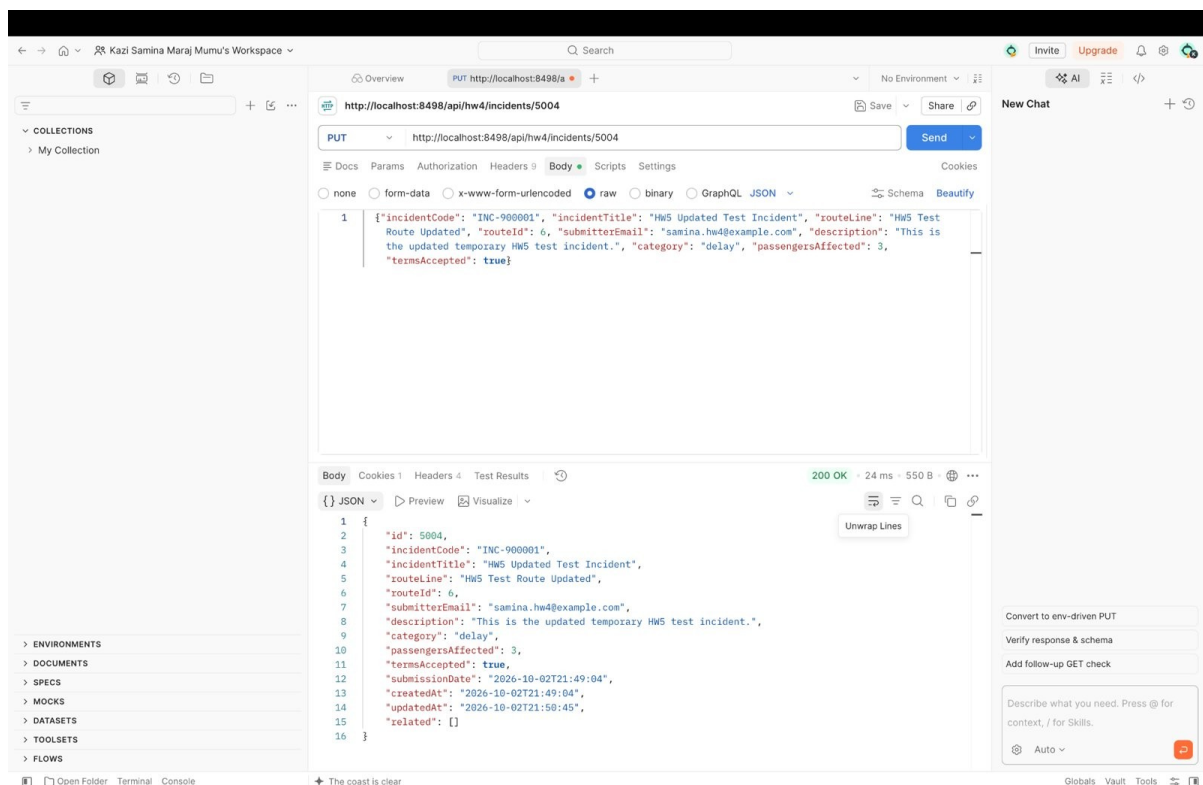


Figure 10. Update the incident (PUT /incidents/5004): new title and passengersAffected 3, 200 OK.

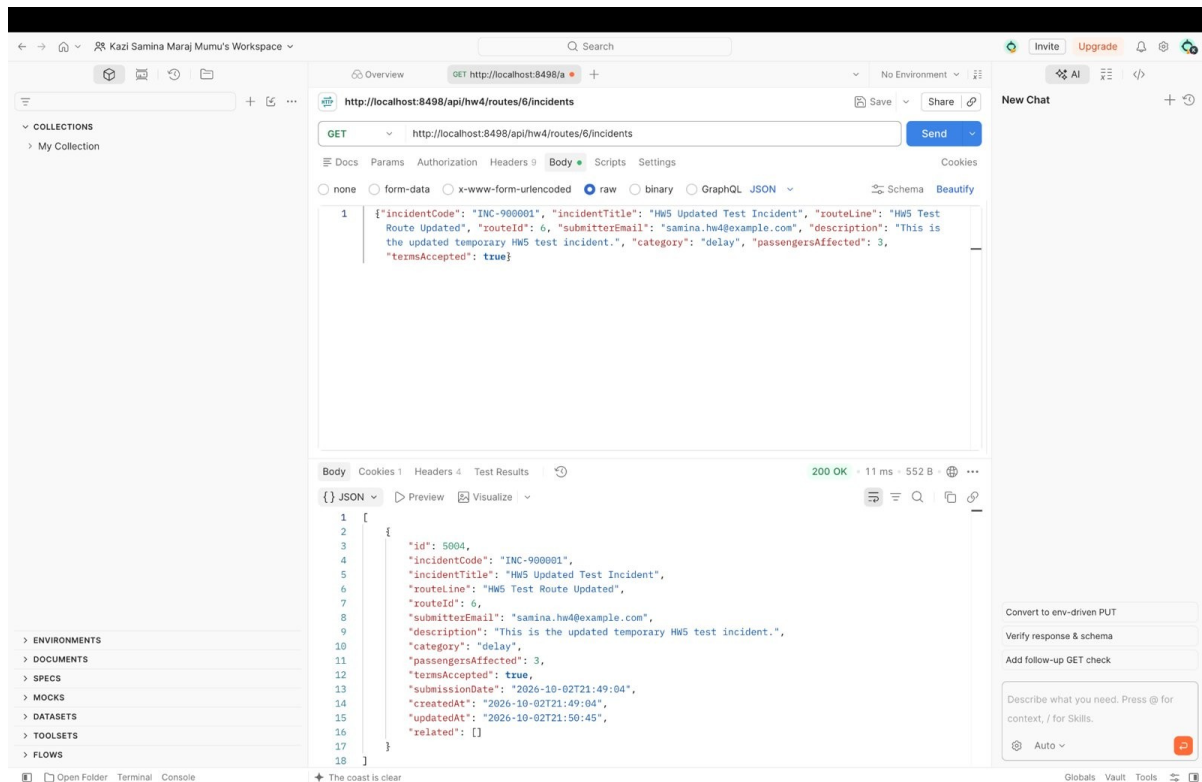


Figure 11. Relationship query (GET /routes/6/incidents) returns only the test incident for route 6.

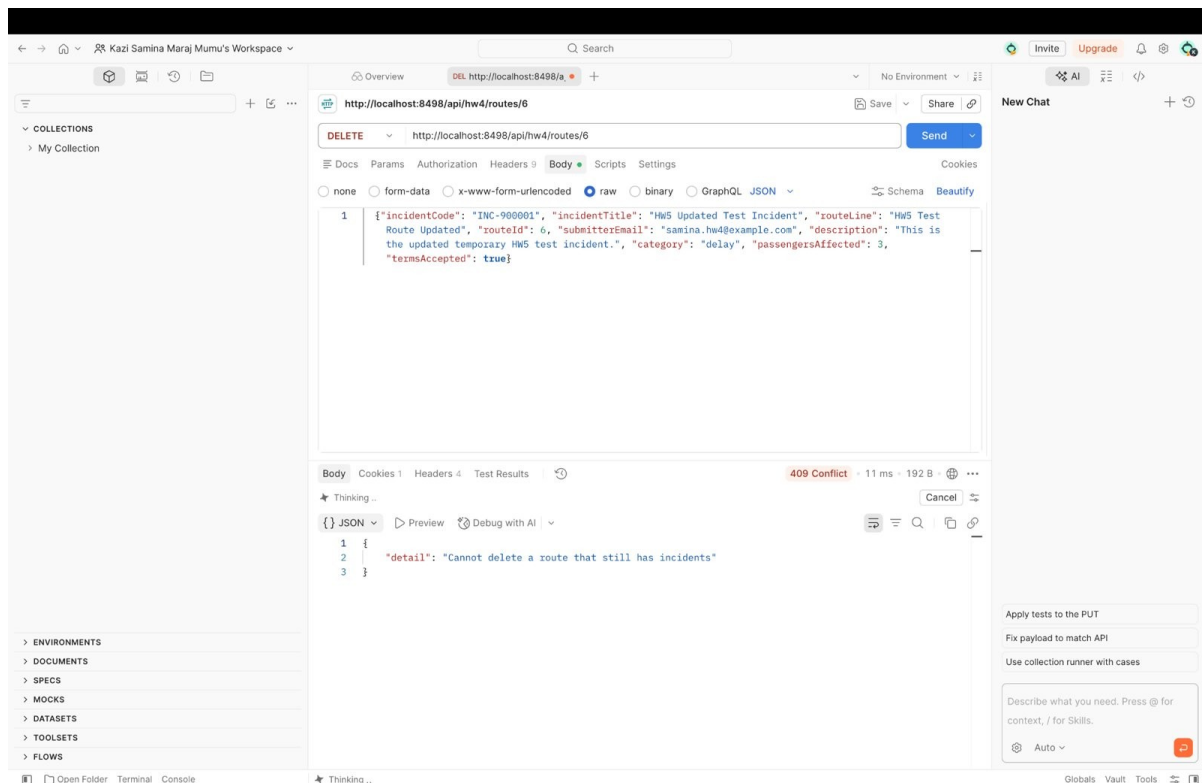


Figure 12. Deleting a route that still has an incident is blocked: 409 Conflict.

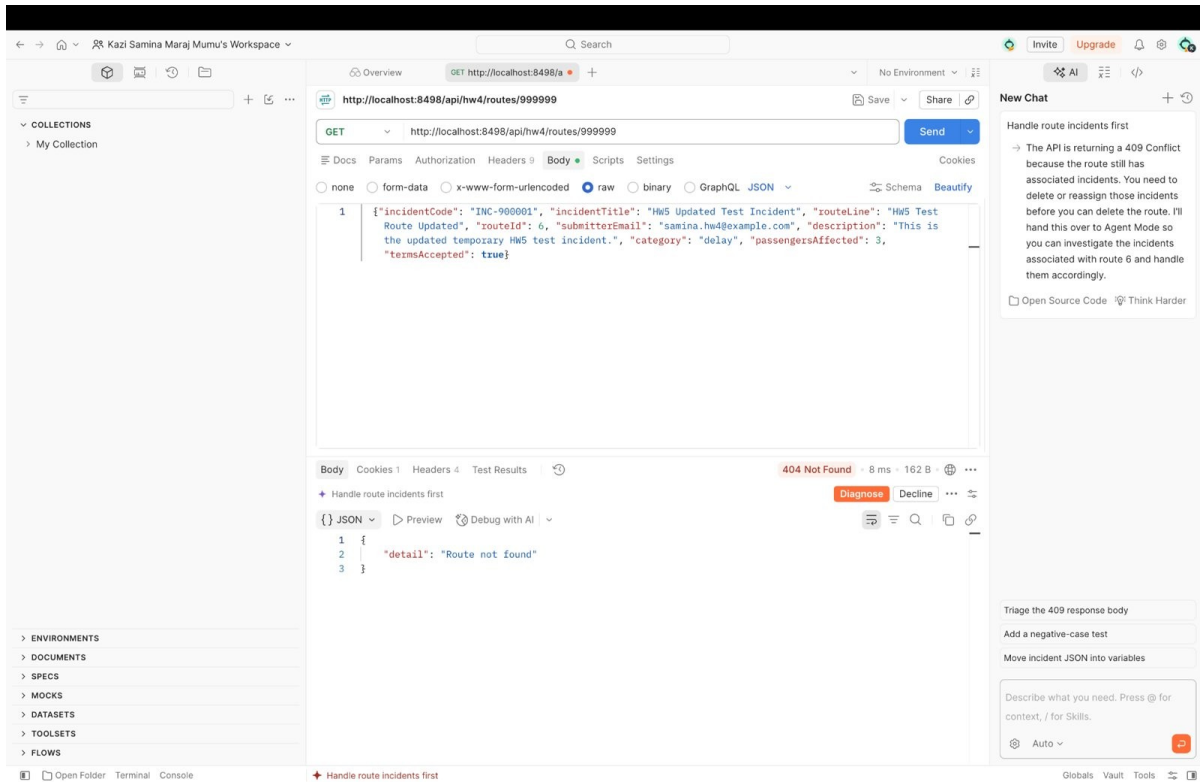


Figure 13. Resource not found: GET /routes/999999 returns 404.

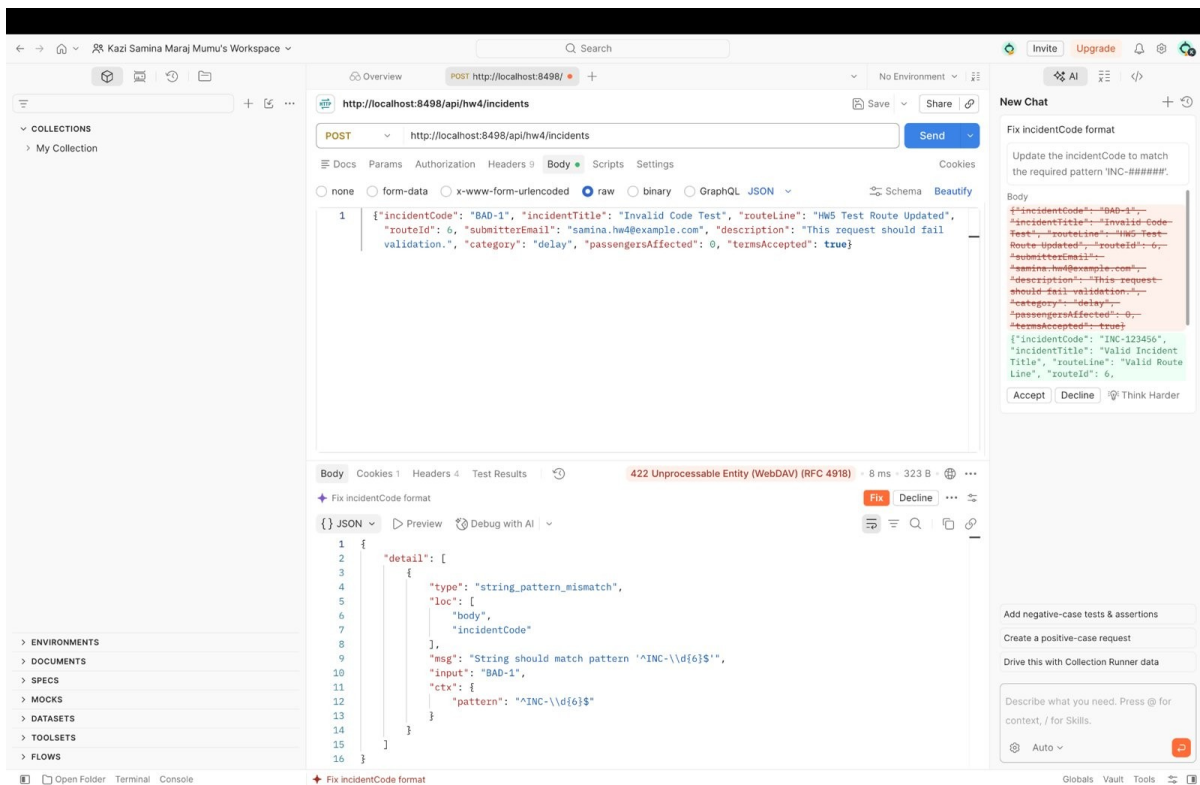


Figure 14. Validation error: incident code BAD-1 does not match `^INC-\\d{6}$`, 422.

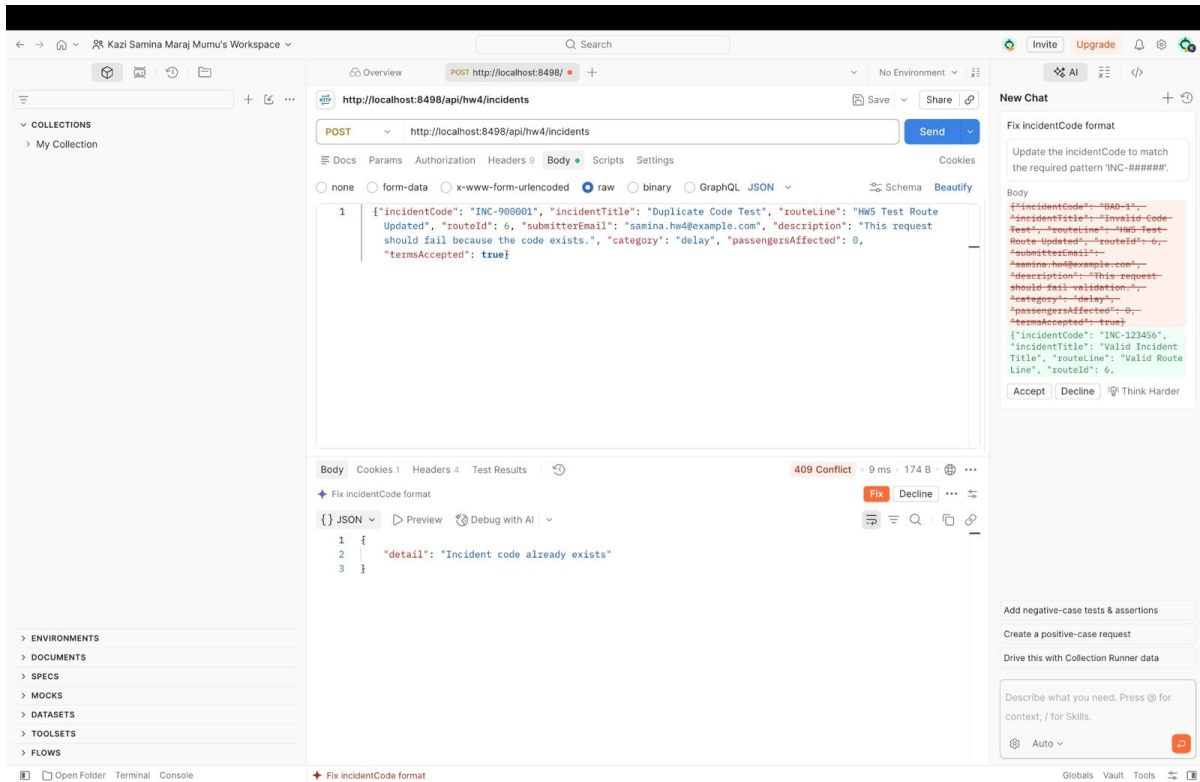


Figure 15. Constraint violation: duplicate incident code INC-900001 returns 409 Conflict.

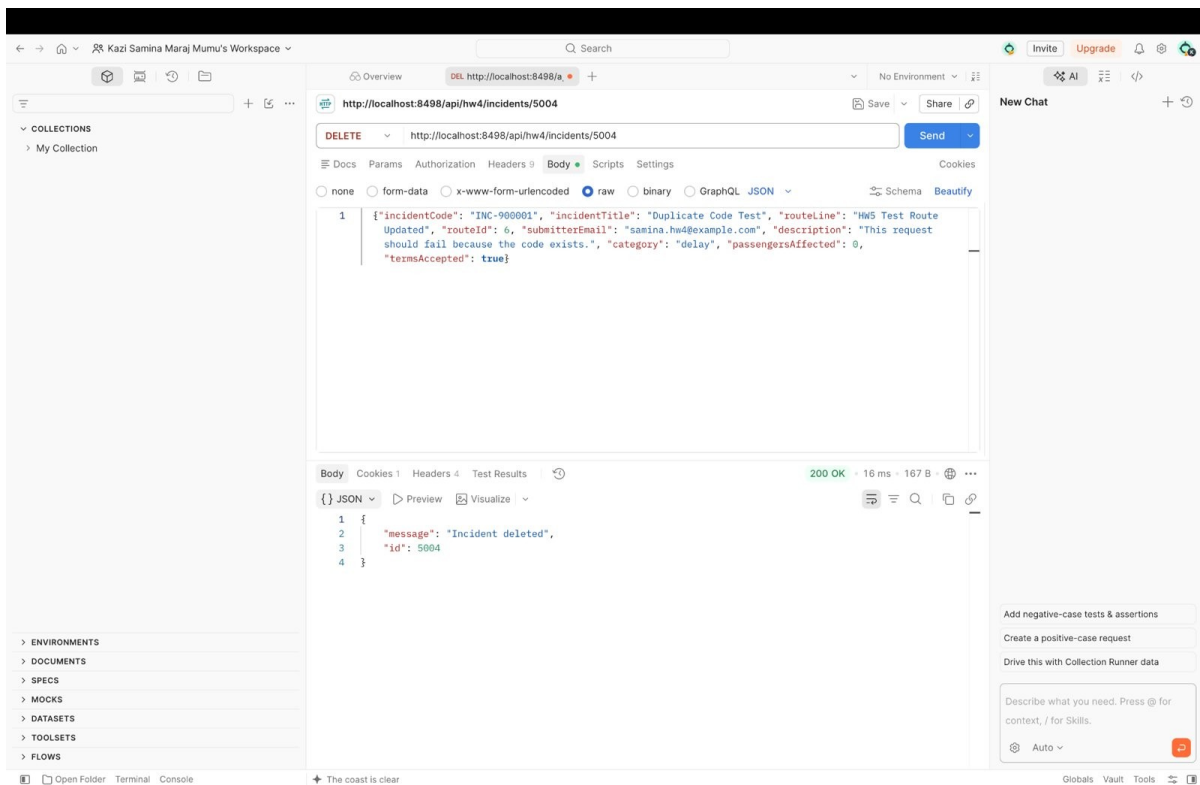


Figure 16. Delete the test incident (DELETE /incidents/5004) returns 200.

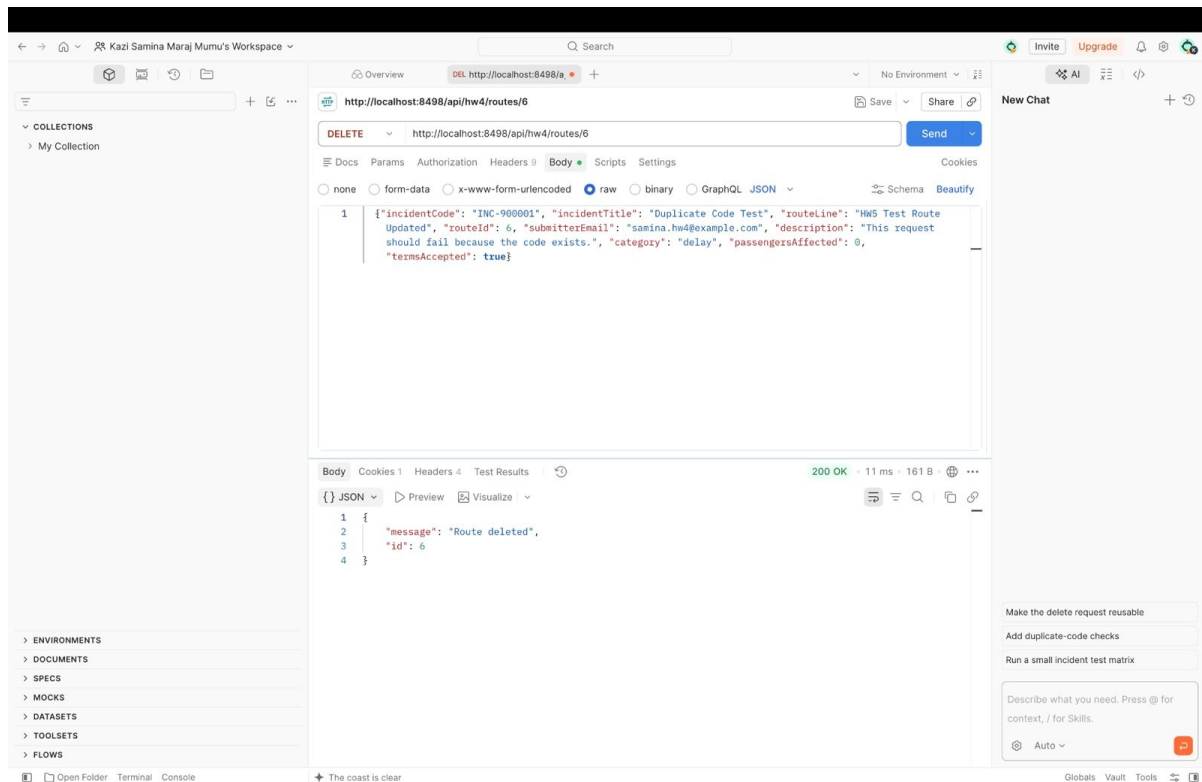


Figure 17. Delete the test route (DELETE /routes/6) now succeeds with 200, because it has no incidents left.

III. Redux Client

I moved the React data layer from local `useState` to Redux Toolkit. The store is in `frontend/src/store/store.js`, and the incidents slice is in `frontend/src/features/incidents/incidentsSlice.js`. The slice keeps the incident list, a loading status and an error message. It has four async thunks that call my FastAPI endpoints through `axios` (with `Credentials` so the session cookie is sent): `fetchIncidents` (GET), `createIncident` (POST), `updateIncident` (PUT /{id}) and `deleteIncident` (DELETE /{id}). Each thunk returns the response on success and uses `rejectWithValue` on failure, and the extraReducers update the list when a thunk is fulfilled.

Example thunk - `frontend/src/features/incidents/incidentsSlice.js`

```
export const createIncident = createAsyncThunk(
  "incidents/createIncident",
  async (payload, thunkAPI) => {
    try {
      const response = await api.post("/incidents", payload);
      return response.data;
    } catch (error) {
      return thunkAPI.rejectWithValue(
        errorMessage(error, "Could not create incident."),
      );
    }
  },
);
```

While testing the Create screen, the form first showed only "Could not save incident." I changed `errorMessage` so it also shows FastAPI validation details. That showed the real problem: the form was not sending the submitter email (the box only displayed the login email), and the route was only sent if I changed the dropdown. I fixed the email in `App.jsx` and selected the route explicitly. I also changed the incident list to show newest records first, so a new incident appears at the top of the Home screen.

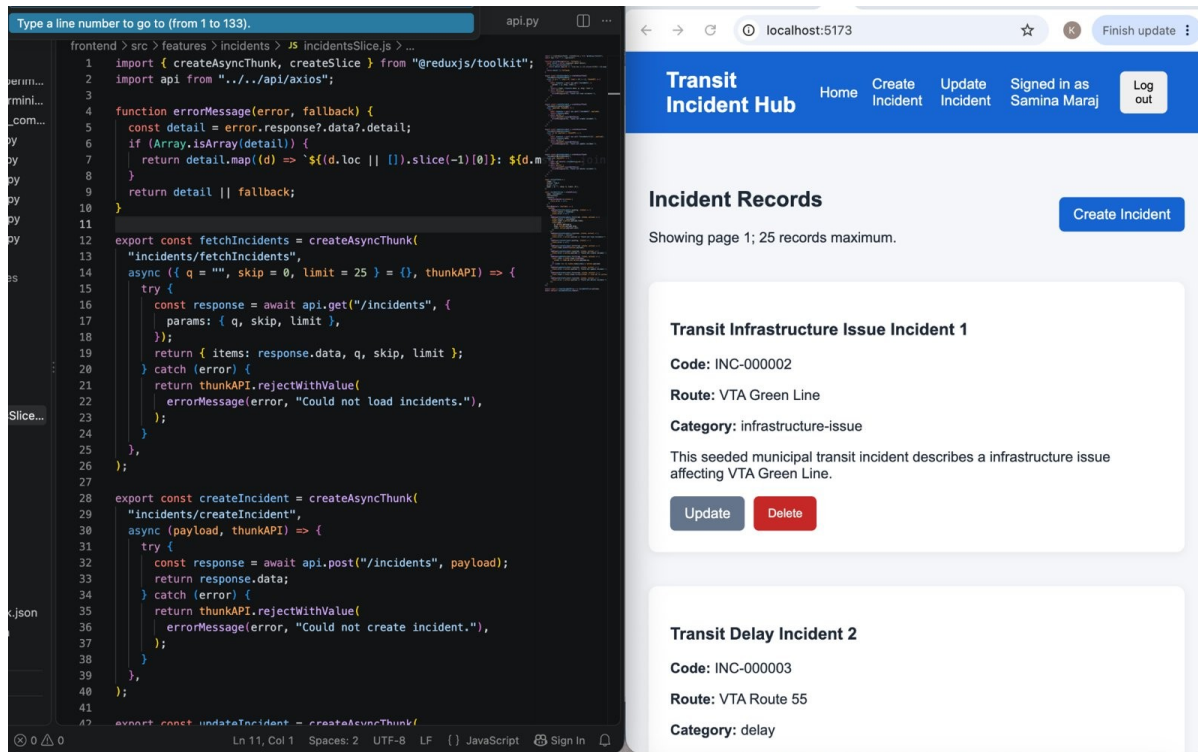


Figure 18. Home: `fetchIncidents` thunk (left) and the Home screen reading the incident list from Redux state (right).

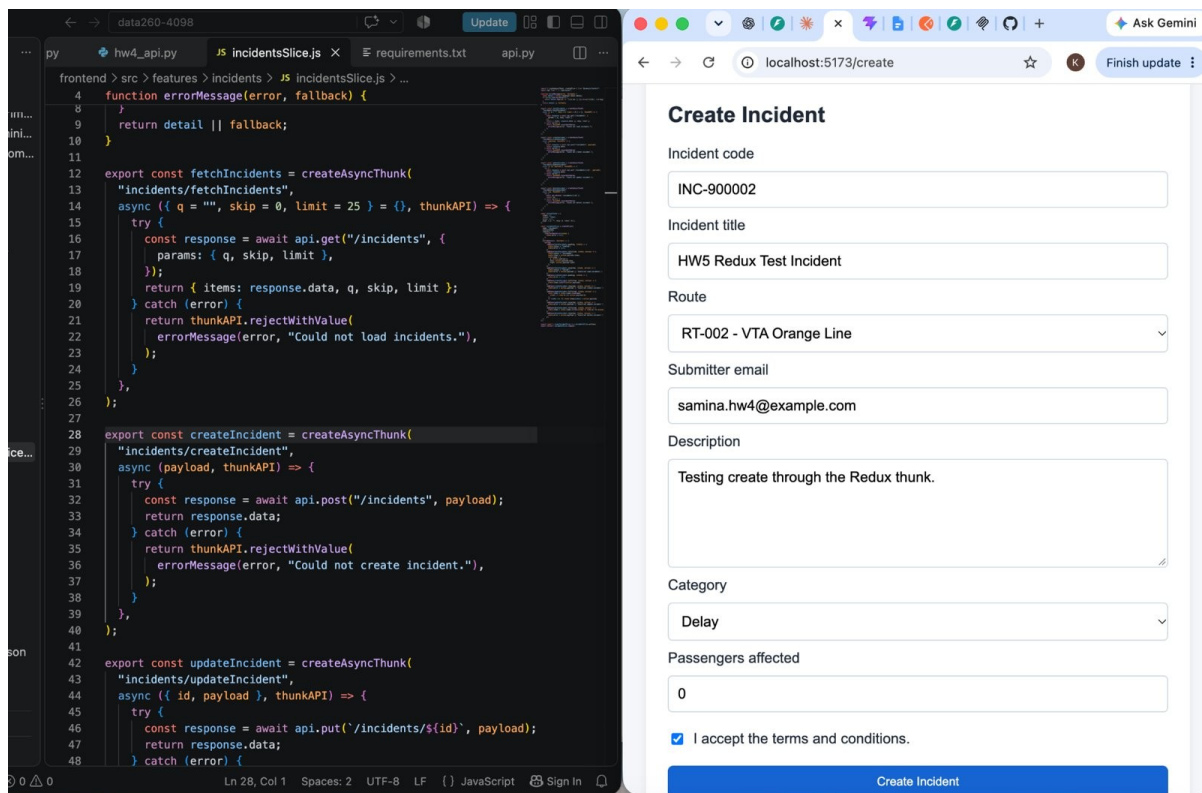


Figure 19. Create: createIncident thunk (left) and the Create form for INC-900002 (right).

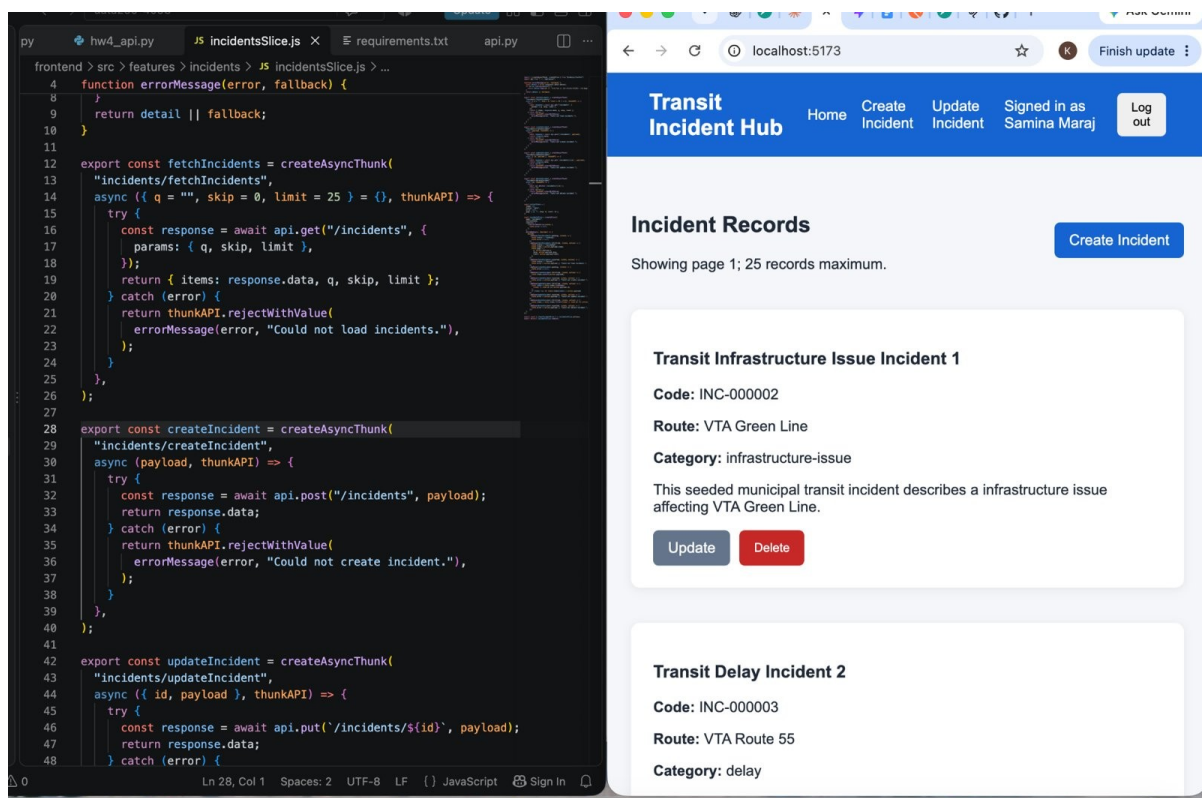


Figure 20. Create: after submitting, the thunk succeeded and the app returned to the Home screen. At this point the list was sorted oldest first, so the new record was on a later page. After I changed the list to newest first, INC-900002 appears at the top of the Home screen (Figure 22).

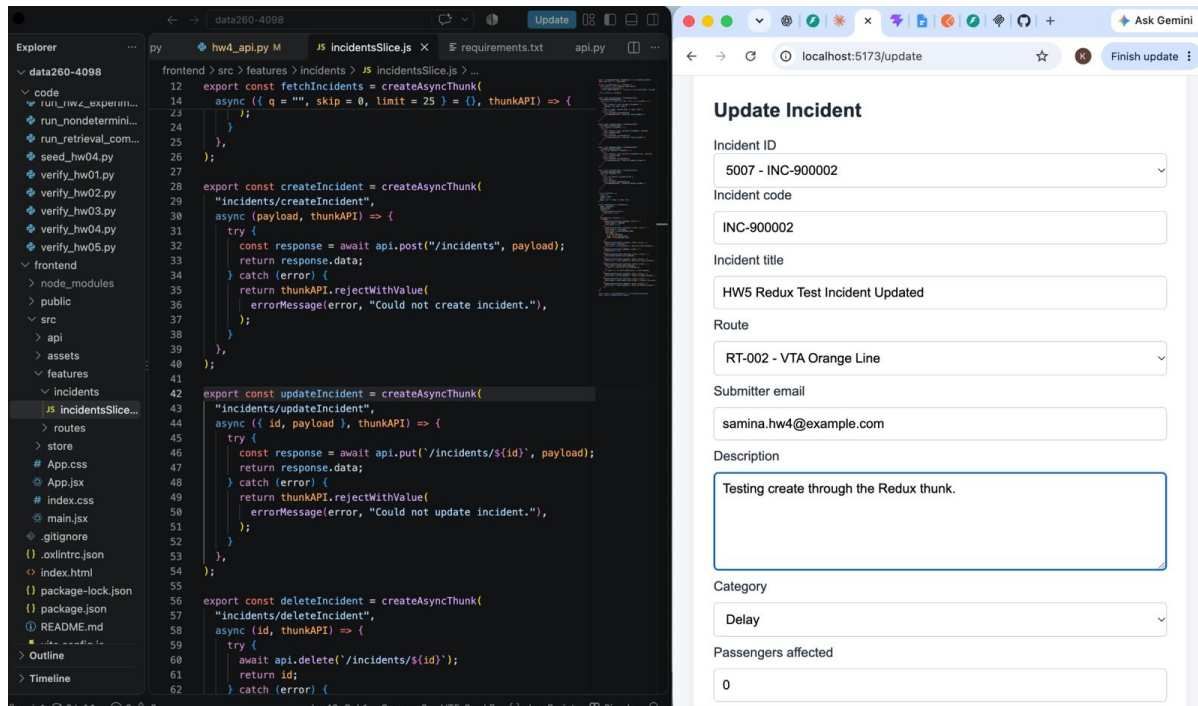


Figure 21. Update: updateIncident thunk (left) and the Update form with incident 5007 selected by ID and a new title (right).

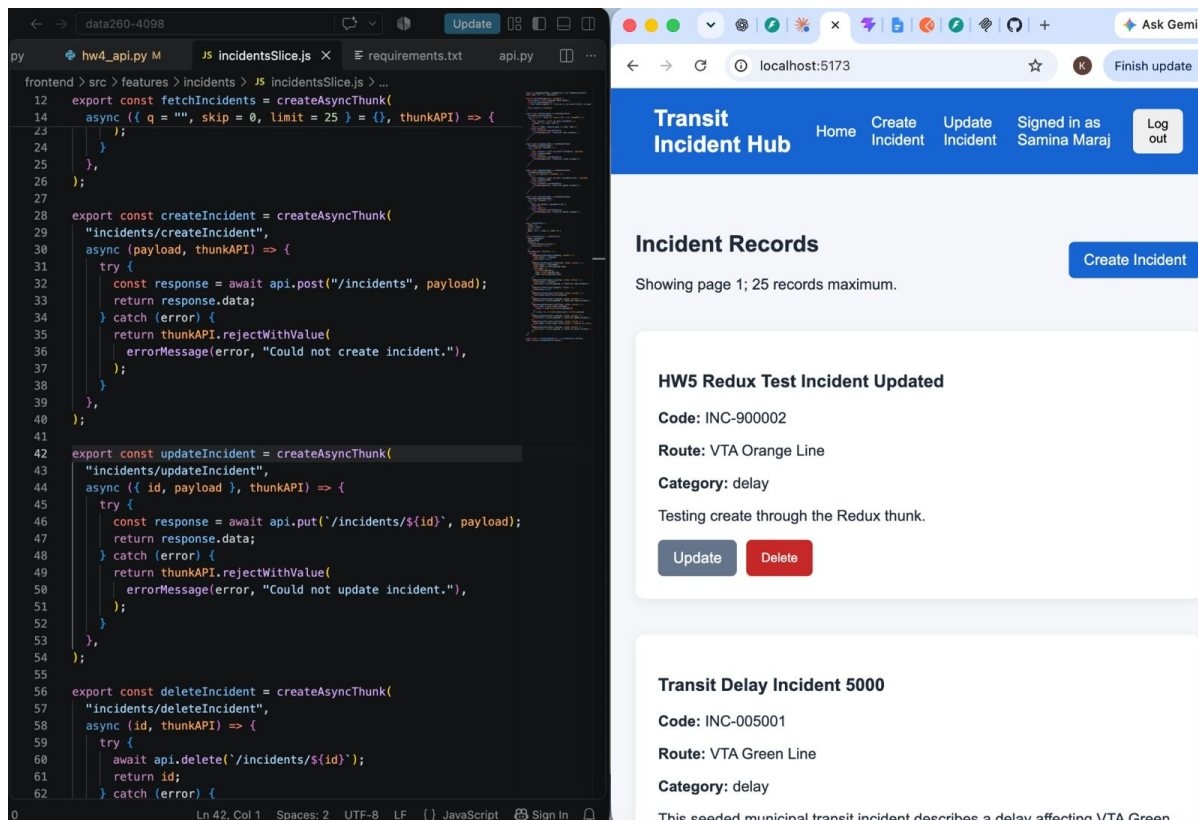


Figure 22. Update: after submitting, the Home list shows "HW5 Redux Test Incident Updated" at the top.

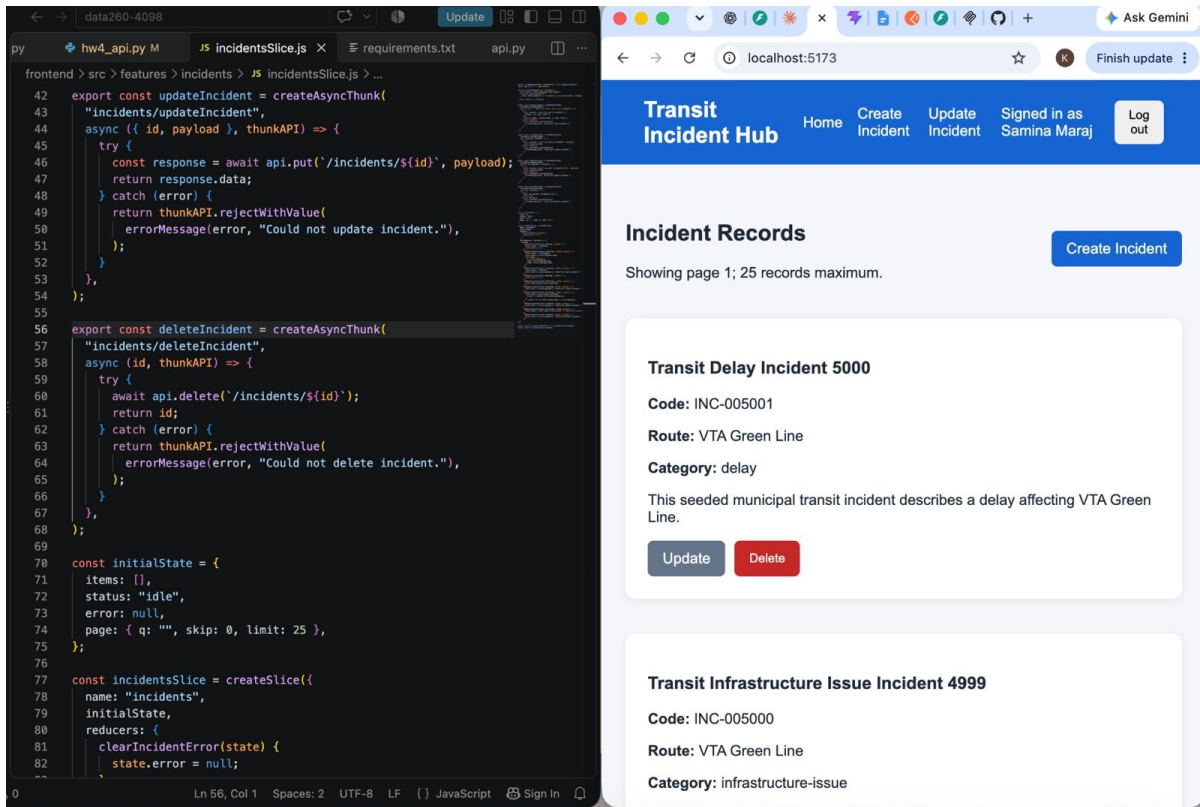


Figure 23. Delete: deleteIncident thunk (left). After clicking Delete, the test incident is removed from the list (right).

Part 2 - MCP Tool Servers

A. TheMealDB Tutorial Server

I built code/mcp/meals_server.py with FastMCP over STDIO. The server is named meals and has four tools that call the public TheMealDB API with the test key 1. All logging goes to stderr, because writing to stdout would break the JSON-RPC stream. If the API returns "meals": null, the tool returns an empty result with a "no matches" message, and network or JSON errors are returned as a clean error.

Tool	Input	Output
search_meals_by_name	query, limit (1-25)	list of {id, name, area, category, thumb}
meals_by_ingredient	ingredient, limit	list of {id, name, thumb}
meal_details	id	{id, name, category, area, instructions, image, source, youtube, ingredients}
random_meal	none	same shape as meal_details

Code - code/mcp/meals_server.py, search_meals_by_name

```
@mcp.tool()
def search_meals_by_name(query: str, limit: int = 5):
    """Search TheMealDB by meal name."""
```

```

query = query.strip()
if not query:
    raise ValueError("query must not be empty")
_check_limit(limit)

payload = _request("search.php", {"s": query})
meals = payload.get("meals")
if meals is None:
    return _no_matches()
return [_summary(meal) for meal in meals[:limit]]

```

One problem I found in MCP Inspector: `search_meals_by_name` failed with "Input should be a valid dictionary". The function returned a list, but its return type was written as dict, so FastMCP rejected the result. I removed the wrong return type from the two list tools and restarted the server, and then they worked.

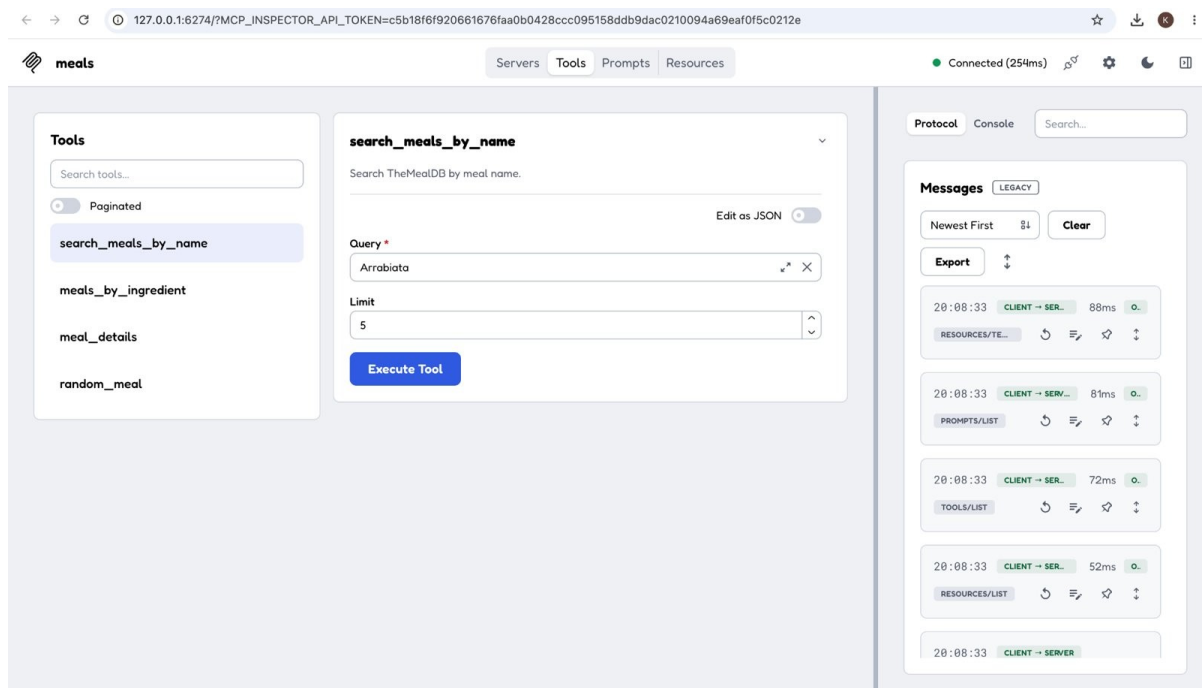


Figure 24. MCP Inspector input: `search_meals_by_name` with query "Arrabiata" and limit 5.

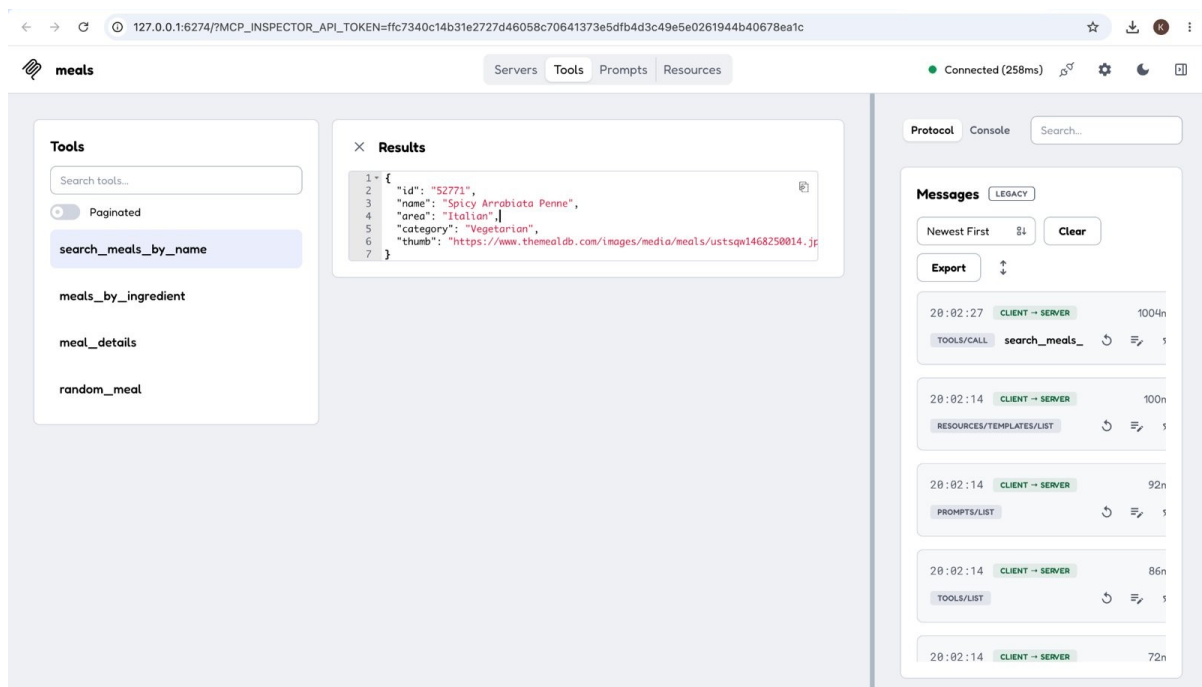


Figure 25. Output: Spicy Arrabiata Penne with id, name, area, category and thumb.

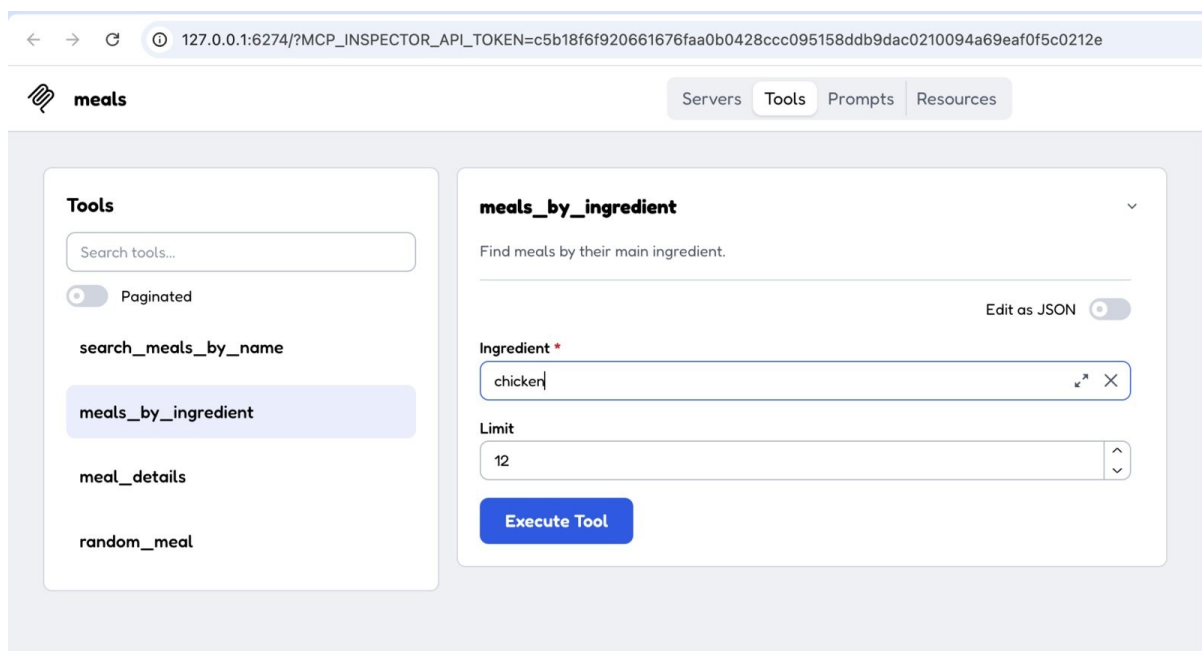


Figure 26. MCP Inspector input: meals_by_ingredient with ingredient "chicken" and limit 12.

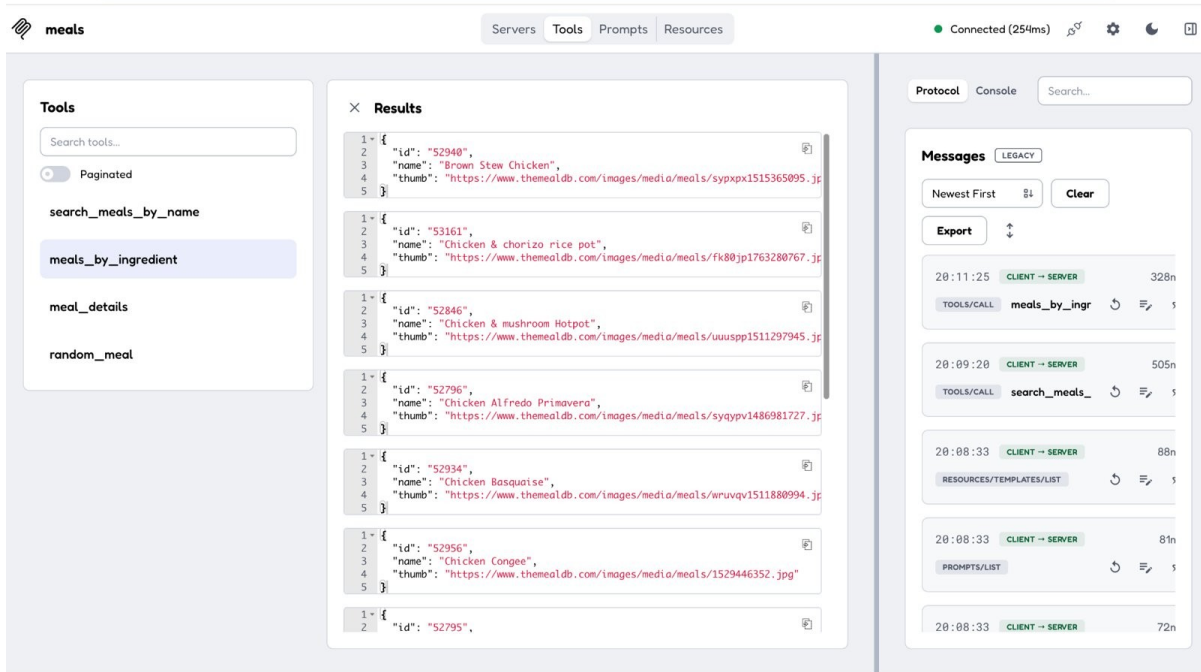


Figure 27. Output: small cards with id, name and thumb for chicken meals.

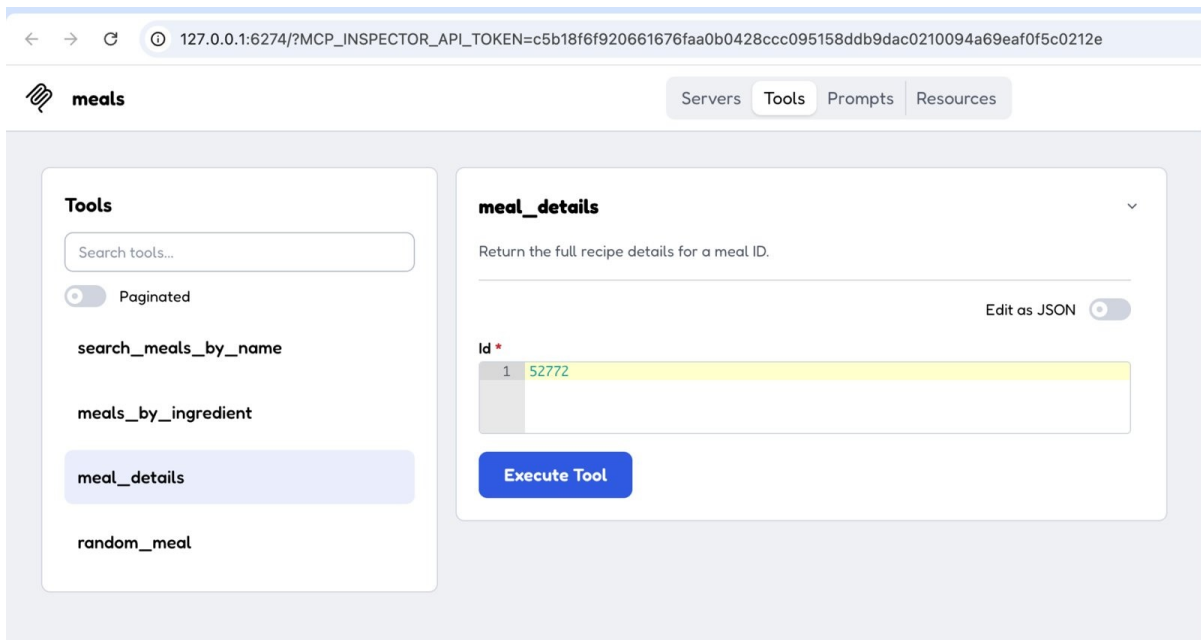


Figure 28. MCP Inspector input: meal_details with id 52772.

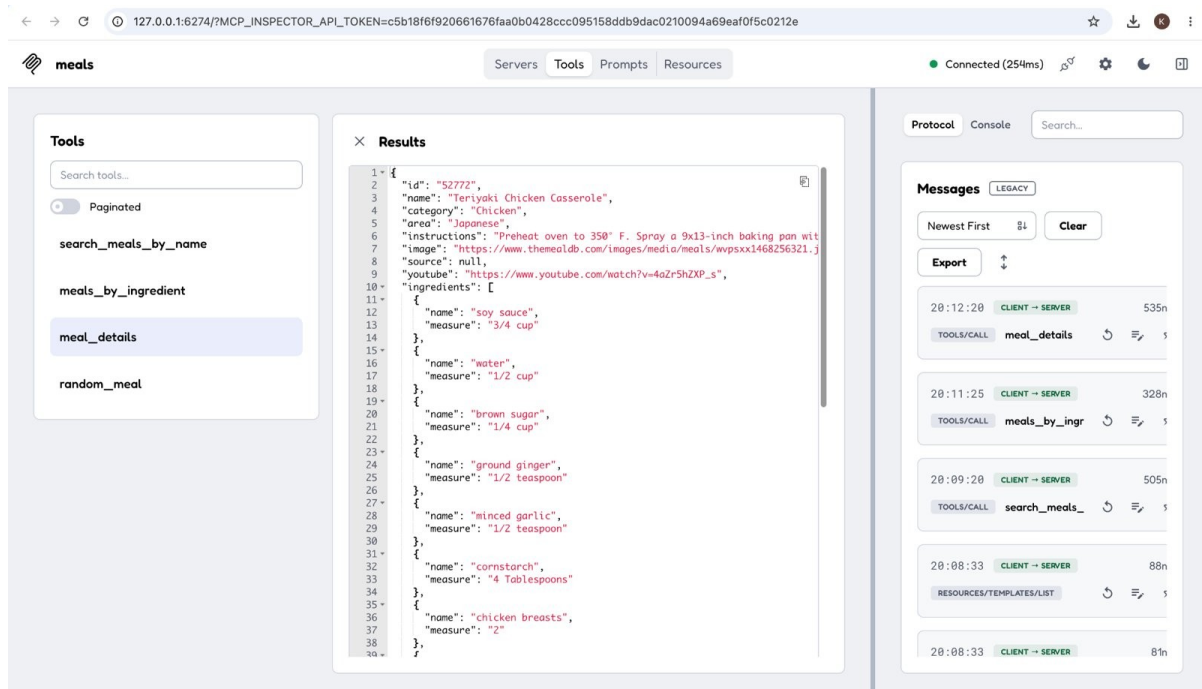


Figure 29. Output: full recipe for Teriyaki Chicken Casserole, including the ingredients list.

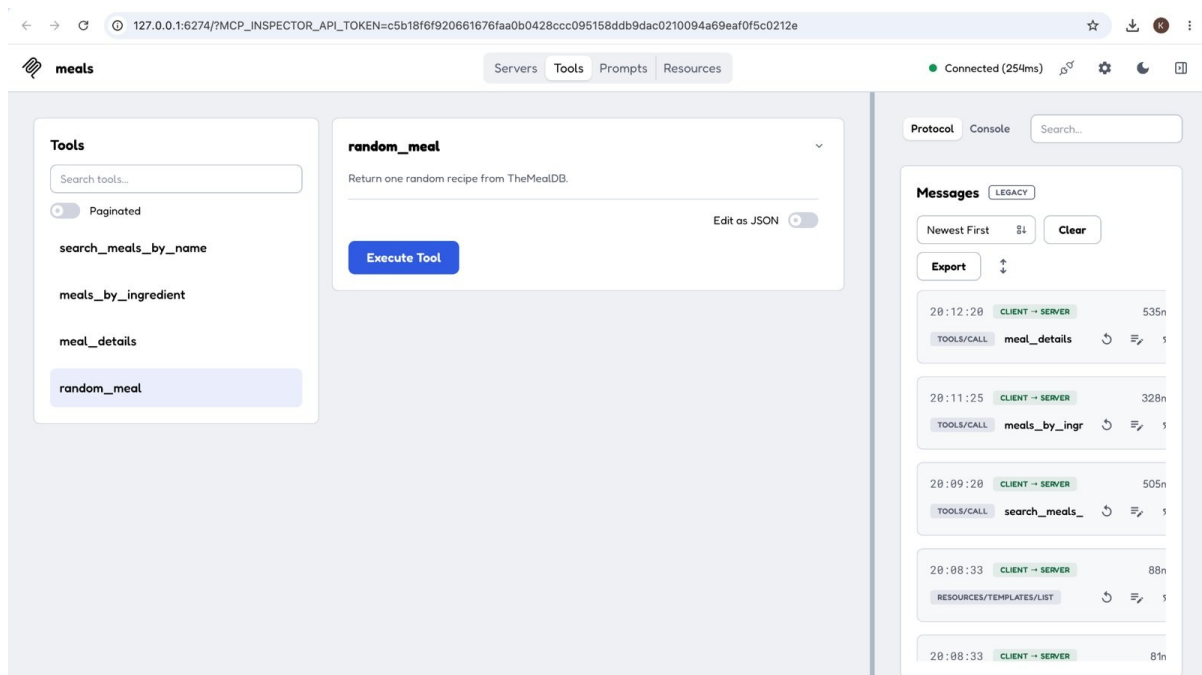


Figure 30. MCP Inspector input: random_meal (no input).

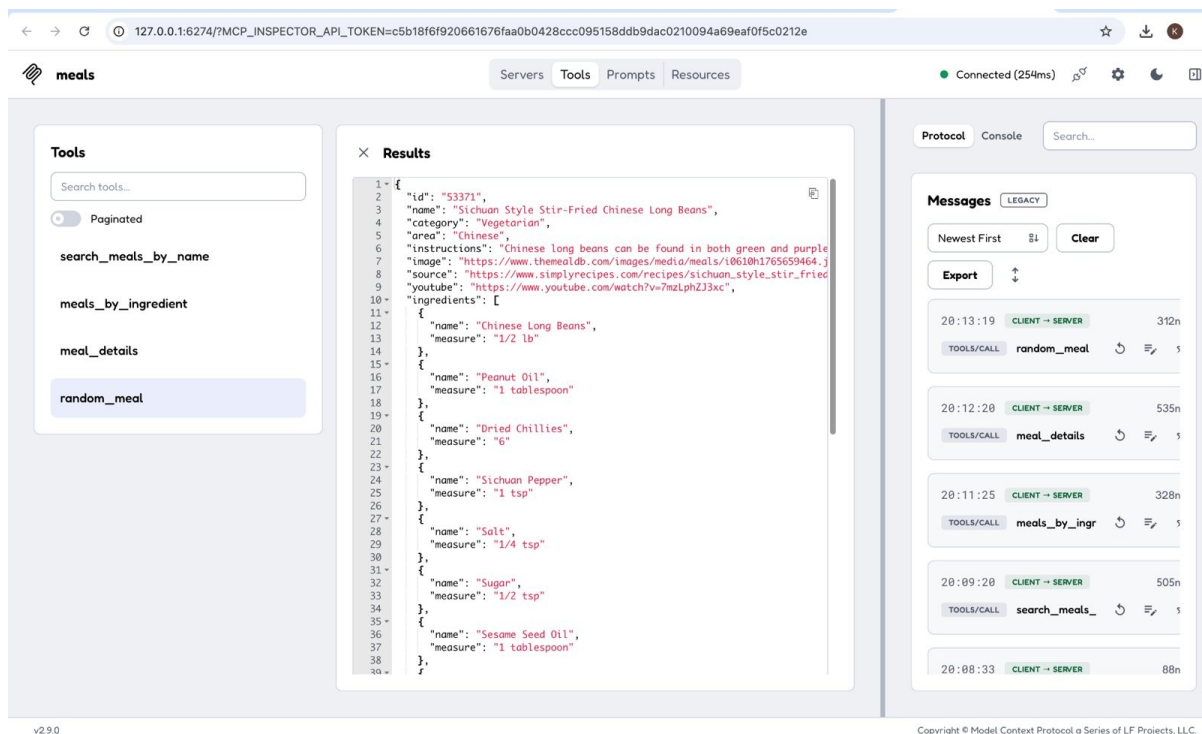


Figure 31. Output: a random meal in the same shape as `meal_details`.

B. Domain MCP Server

The second server, `code/mcp/transit_server.py`, works over my `s4098_rel` database and has exactly three tools: `search_incidents` (search), `get_incident` (detail lookup) and `incident_count_by_route` (aggregate). All three return the same envelope, `{ok, data, error}`, which is defined once in `code/mcp/envelope.py`. On success error is null, and on failure data is null. This is the same envelope I reuse in `execute_tool` in Part 4.

Code - `code/mcp/envelope.py` (shared `{ok, data, error}` envelope)

```

def tool_envelope(
    data: Any = None,
    error: str | None = None,
) -> dict[str, Any]:
    """Return the one response shape used by every domain tool."""
    return {
        "ok": error is None,
        "data": data if error is None else None,
        "error": error,
    }

```

Code - `code/mcp/transit_server.py`, input validation in `get_incident` and `incident_count_by_route`

```

@mcp.tool()
def get_incident(incident_code: str) -> dict[str, Any]:
    """Look up one incident by its validated incident code."""
    if not INCIDENT_CODE_PATTERN.fullmatch(incident_code.strip()):
        return _error("incident_code must match INC-#####")
    # ...

@mcp.tool()
def incident_count_by_route(min_incidents: int = 0) -> dict[str, Any]:
    """Count incidents by Route, optionally filtering by a minimum count."""

```

```
if min_incidents < 0:
    return _error("min_incidents must be greater than or equal to 0")
# ...
```

When I first opened this server in MCP Inspector, it failed with "No module named 'code.mcp'". My project folder is called code, which clashes with Python's built-in code module, and Inspector does not pass PYTHONPATH. I fixed it by importing envelope directly and adding the project root to sys.path.

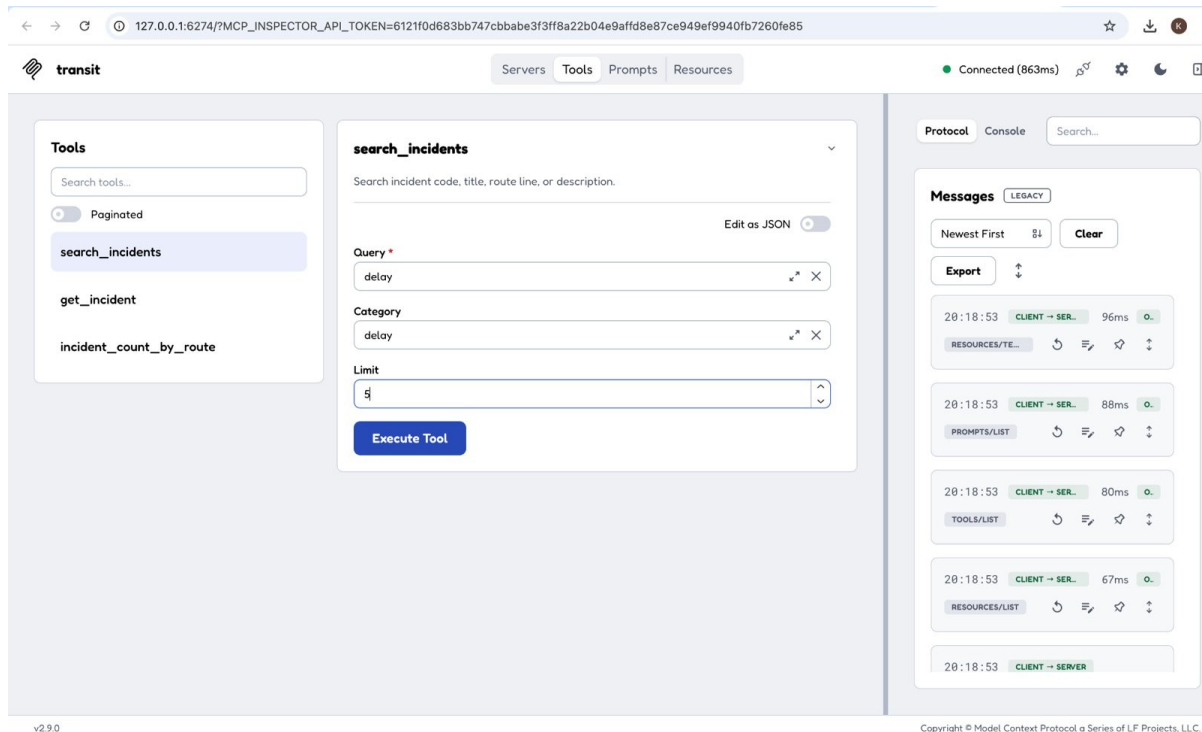


Figure 32. *search_incidents* valid input: query "delay", category "delay", limit 5.

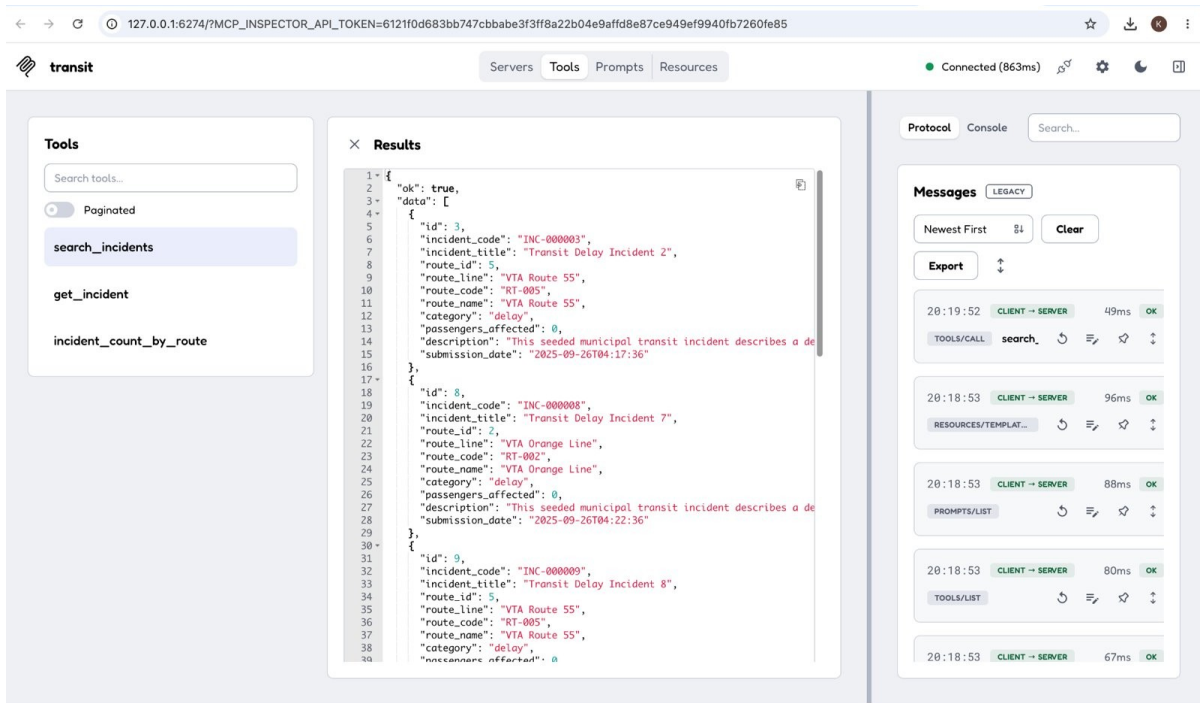


Figure 33. `search_incidents` valid output: `ok true` with five delay incidents.

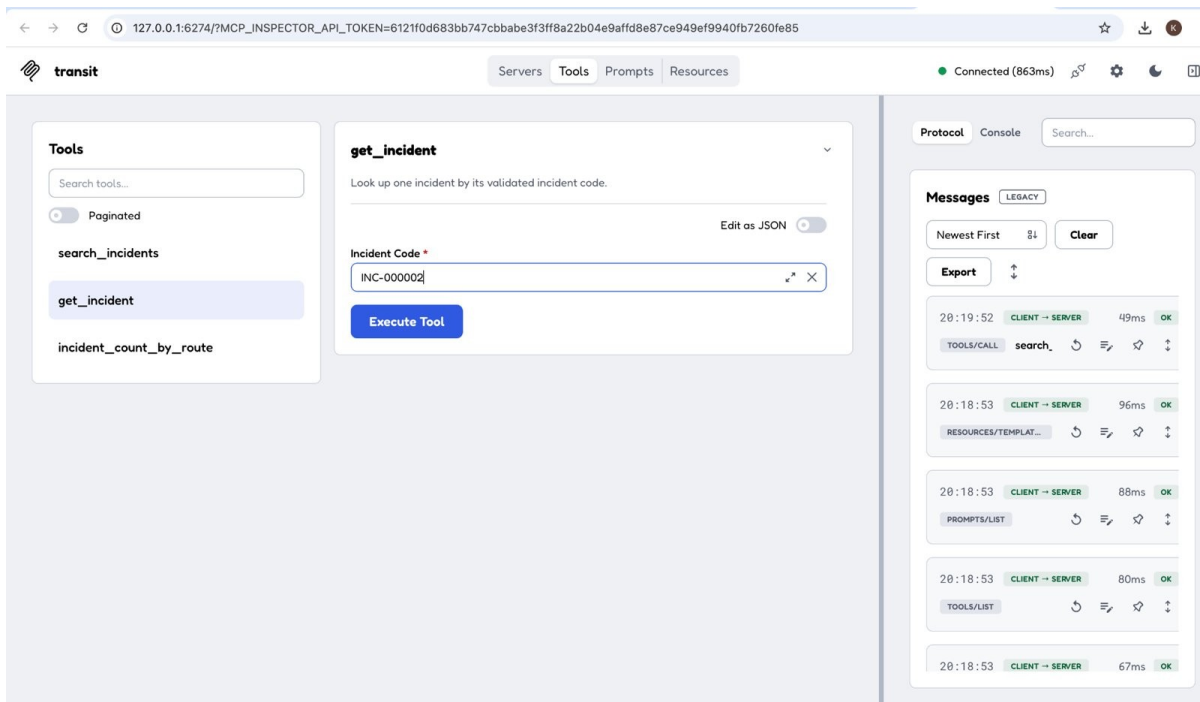


Figure 34. `get_incident` valid input: `INC-000002`.

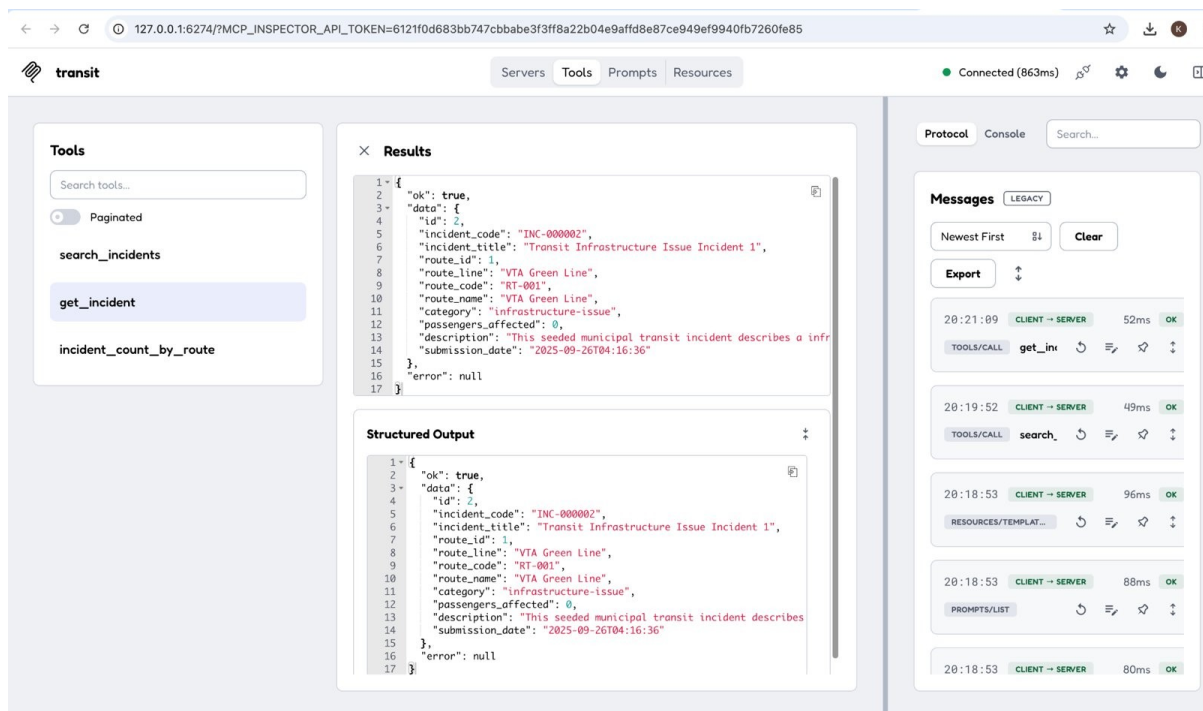


Figure 35. `get_incident` valid output: `ok true` with the incident details and error `null`.

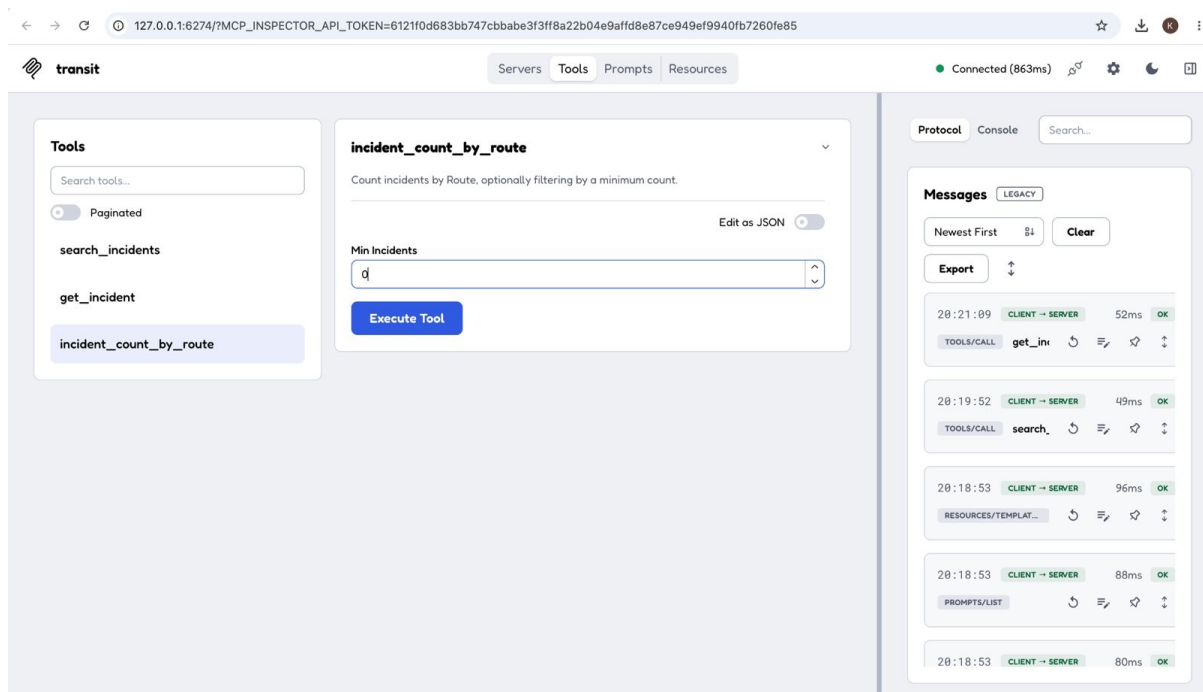


Figure 36. `incident_count_by_route` valid input: `min_incidents 0`.

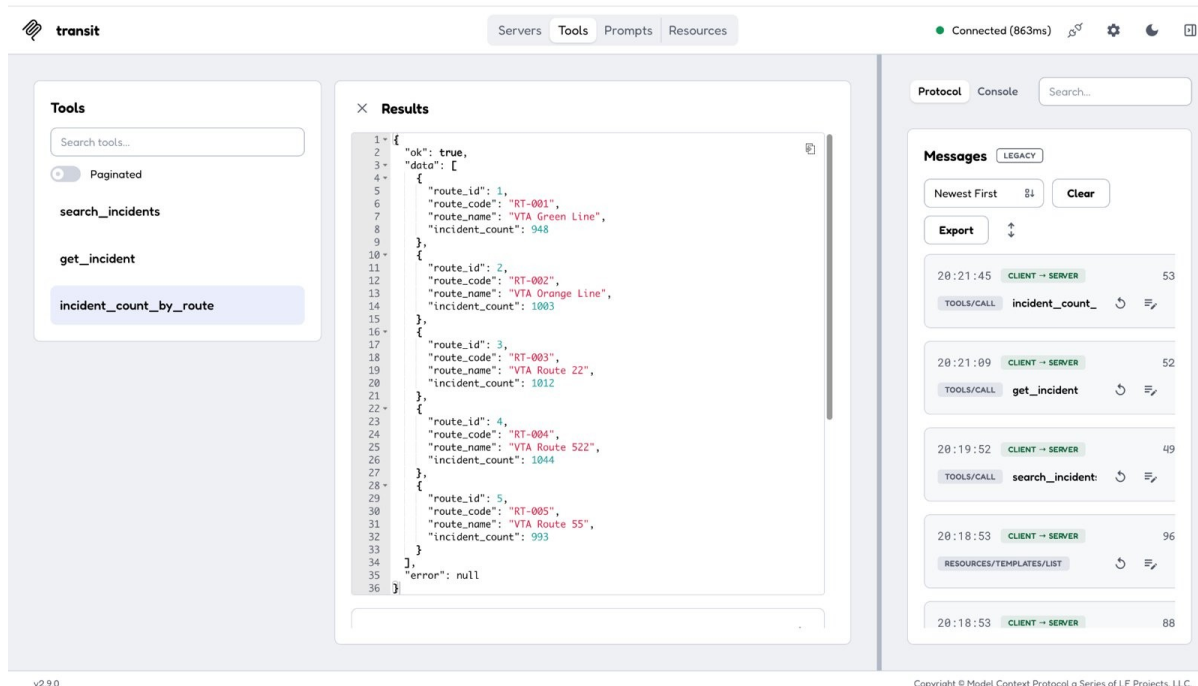


Figure 37. `incident_count_by_route` valid output: incident counts for all five routes.

I also ran one intentionally invalid call for each tool. Each was rejected with `ok` false and a clear error, without crashing the server:

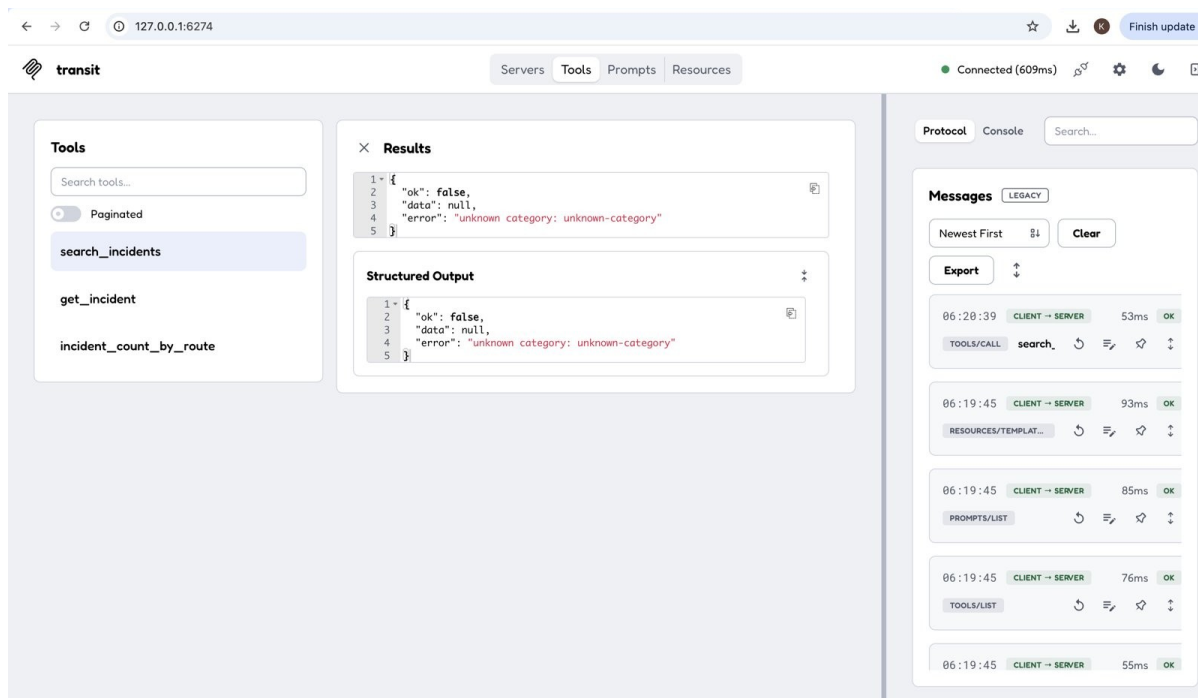


Figure 38. `search_incidents` invalid call: category "unknown-category" returns `{"ok": false, "data": null, "error": "unknown category: unknown-category"}`.

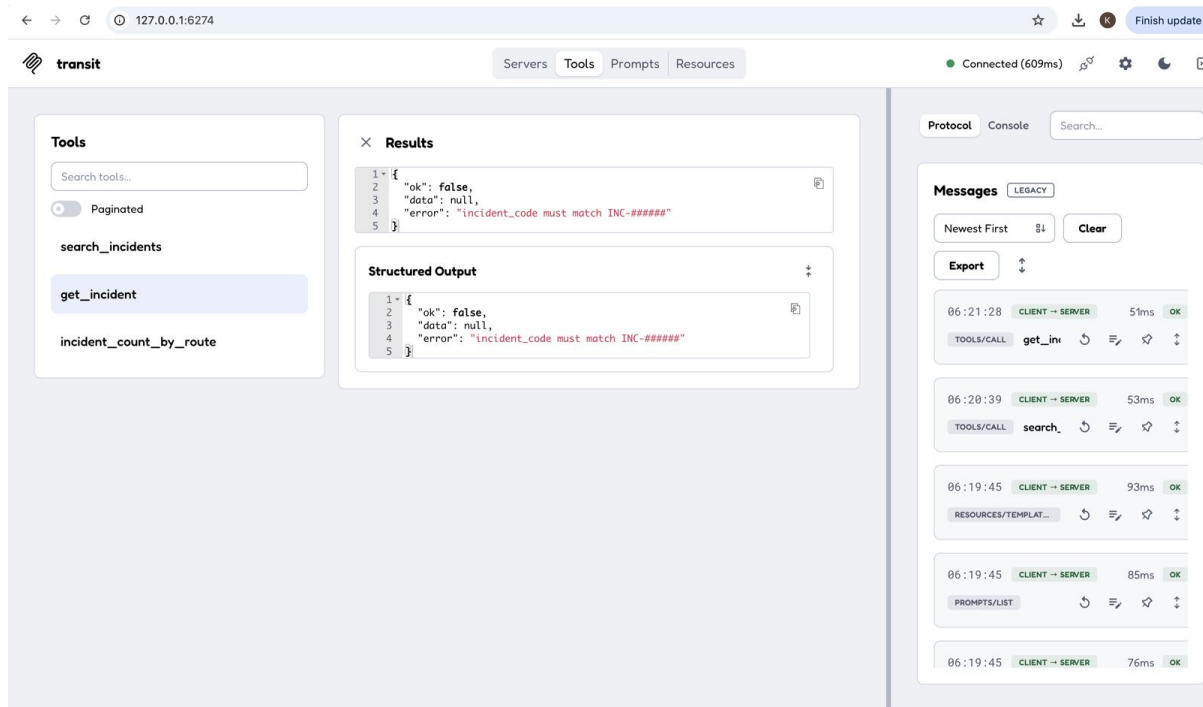


Figure 39. *get_incident* invalid call: *incident_code* "abc" returns {"ok": false, "data": null, "error": "incident_code must match INC-#####"}.

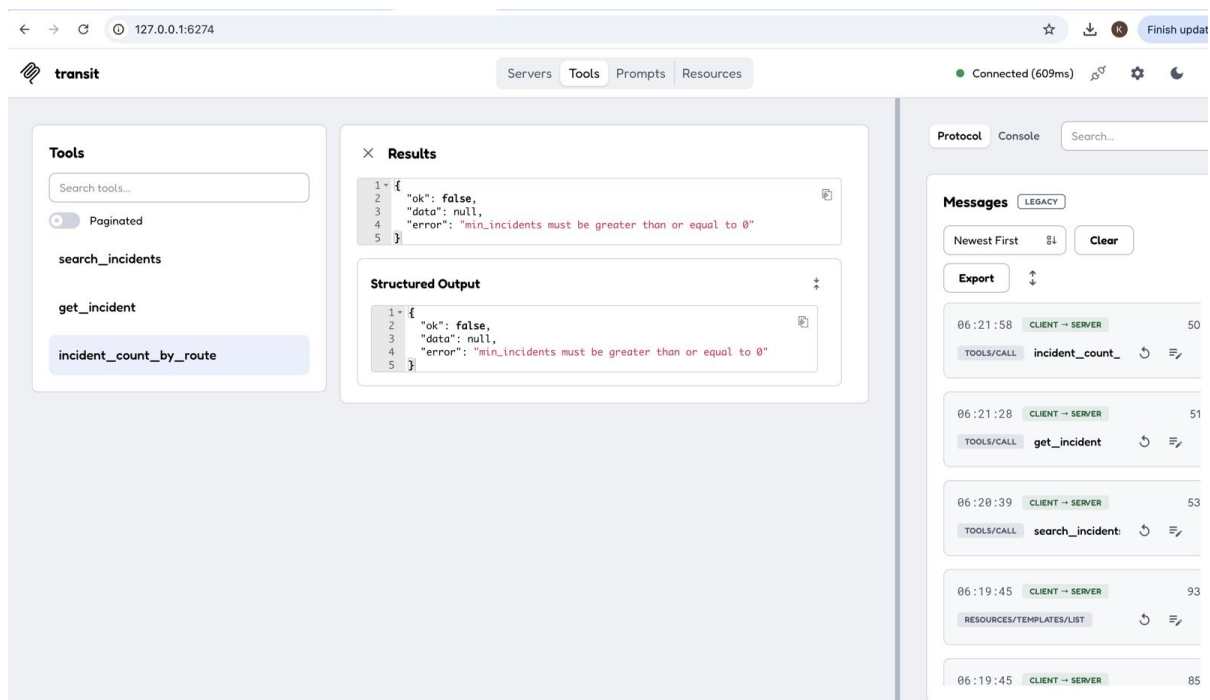


Figure 40. *incident_count_by_route* invalid call: *min_incidents* -5 returns {"ok": false, "data": null, "error": "min_incidents must be greater than or equal to 0"}.

The outputs of all 10 tool calls (4 meals tools, plus valid and invalid calls for the 3 transit tools) are saved as JSON in reports/hw05/raw/inspector/.

Part 3 - Tool Contracts Under Stress

1. Rejected Calls from Part 2B

Tool	Expected input schema	Rejected input	Returned error	Why it was rejected
search_incidents	query: string, category: one of delay / collision / service-suspension / infrastructure-issue (optional), limit: int (at most 50)	{"query": "delay", "category": "unknown-category", "limit": 10}	unknown category: unknown-category	The category is not one of the allowed values.
get_incident	incident_code: string matching INC-#####	{"incident_code": "abc"}	incident_code must match INC-#####	"abc" does not match the incident code format.
incident_count_by_route	min_incidents: int >= 0 (default 0)	{"min_incidents": -5}	min_incidents must be greater than or equal to 0	A count filter cannot be negative.

All three were returned as {"ok": false, "data": null, "error": "...". The full table is also in reports/hw05/CONTRACTS.md.

2. Timeouts and Bounded Exponential Backoff

I added a retry wrapper (code/mcp/retry_utils.py) around the database operations in the domain tools. Each attempt has a timeout, and the delay between attempts doubles but is capped.

Setting	Value
max_retries	2 (so at most 3 attempts)
Base backoff delay	0.05 s
Maximum delay cap	0.20 s
Per-attempt timeout	2.0 s

Code - code/mcp/retry_utils.py, retry_call (per-attempt timeout and bounded exponential backoff)

```
for attempt in range(1, total_attempts + 1):
    try:
        if fault_injector is not None:
            fault_injector()
        value = _run_with_timeout(operation, timeout_seconds)
        return RetryResult(
            success=True,
            value=value,
            attempts=attempt,
            latency_ms=(time.perf_counter() - started) * 1000,
        )
    except Exception as exc: # one clean result for every failed call
        last_error = exc
        if attempt == total_attempts:
            break
```

```

delay = min(
    base_delay_seconds * (2 ** (attempt - 1)),
    max_delay_seconds,
)
time.sleep(delay)

```

I ran `code/retry_demo.py` to show the three required cases:

```

Kazi Samina Maraj Mumu
case=success_first_attempt RetryResult(success=True, value='success', error=None, attempts=1, latency_ms=0.012042000889778137)
case=failure_then_success RetryResult(success=True, value='success after retry', error=None, attempts=2, latency_ms=55.038709004293196)
case=failure_after_all_retries RetryResult(success=False, value=None, error='demo injected failure', attempts=3, latency_ms=160.09624999423977)
((.venv) ) (base) samina@Kazis-MacBook-Pro-2 data260-4098 %

```

Figure 41. `code/retry_demo.py`: success on the first attempt (1 attempt), failure then success after a retry (2 attempts, about 55 ms), and failure after all 3 attempts returning a clean error instead of crashing (about 160 ms).

3. Seeded Fault Injection

For fault injection I used `random.Random(264098)`, my `VERIFY_SEED`, so the same seed always gives the same success/failure sequence. I ran 50 calls at each failure rate (150 calls in total). All 150 records are saved in `reports/hw05/raw/fault_injection.csv`. To check reproducibility, I ran the experiment a second time and compared the success/failure columns of both runs, and they were the same sequence.

Code - `code/mcp/retry_utils.py`, `SeededFaultInjector` (seeded with `VERIFY_SEED 264098`)

```

class SeededFaultInjector:
    """Produce a repeatable failure sequence for a selected failure rate."""

    def __init__(self, failure_rate: float, seed: int = VERIFY_SEED):
        if not 0.0 <= failure_rate <= 1.0:
            raise ValueError("failure_rate must be between 0.0 and 1.0")
        self.failure_rate = failure_rate
        self.rng = random.Random(seed)

    def __call__(self) -> None:
        if self.rng.random() < self.failure_rate:
            raise InjectedFailure(
                f"seeded injected failure at rate {self.failure_rate:.0%}"
            )

```

Injected failure rate	Success rate	Mean latency (ms)	p99 latency (ms)
0%	100% (50/50)	1.216	15.999
20%	98% (49/50)	21.222	165.130
50%	84% (42/50)	59.542	165.596

```

Kazi Samina Maraj Mumu
total_calls=150
rate=0% calls=50 success_rate=1.000 mean_latency_ms=1.216 p99_latency_ms=15.999
rate=20% calls=50 success_rate=0.980 mean_latency_ms=21.222 p99_latency_ms=165.130
rate=50% calls=50 success_rate=0.840 mean_latency_ms=59.542 p99_latency_ms=165.596
((.venv) ) (base) samina@Kazis-MacBook-Pro-2 data260-4098 %

```

Figure 42. Fault-injection results for the 150 seeded calls, from my logged run in `RUN_LOG.txt`.

4. Evaluation

The results match what I expected from the retry policy. With three attempts, a call only fails if all three attempts fail. At a 50% failure rate that is $0.5 \times 0.5 \times 0.5 = 12.5\%$, so about 87.5% of calls should succeed, and I got 84%. At 20% about 99% should succeed, and I got 98%. The p99 latency stays around 165 ms, which is the two backoff delays (50 ms + 100 ms) plus the query time.

For an interactive assistant, I think this policy is mostly suitable. In my runs the p99 latency was about 165 ms, which a user would not really notice, and the retries fixed most temporary failures. However, 165 ms is not the worst case. If every attempt hangs until the 2-second timeout, one call can take about 3×2.0 s plus 0.05 s and 0.10 s of backoff, which is roughly 6.15 seconds. That is too long for a chat reply, so for an interactive assistant I would lower the per-attempt timeout (for example to 0.5-1 s). Also, when failures are frequent (50%), 16% of calls still fail, so the assistant should show a clear error message instead of retrying for a long time.

For batch processing, longer waits are acceptable, so I would change the policy to favor completion over speed. I would allow more retries (for example 5 to 6), use a longer timeout per attempt, and use larger backoff delays with a higher cap (for example starting at 0.5 s and capped at several seconds) with some random jitter, so many jobs do not retry at the same moment. That would push the success rate much closer to 100% at the cost of slower individual calls.

Part 4 - One Safe Tool Entry Point and Offline Tests

`execute_tool(name, inputs)` in `code/mcp/execute_tool.py` is the single function the agent uses to call any of the three domain tools. It reuses the {ok, data, error} envelope from Part 2B, returns the result as a JSON string, and never raises. An unknown tool name, missing inputs, extra inputs or wrong types all come back as ok false.

Code - `code/mcp/execute_tool.py`, `execute_tool`

```
def execute_tool(name, inputs):
    """Validate, dispatch, and JSON-encode one domain-tool call.

    This function deliberately catches every normal application exception and
    always returns a JSON string containing the shared response envelope.
    """
    try:
        validated, validation_error = _validated_inputs(name, inputs)
        if validation_error:
            return json.dumps(tool_envelope(error=validation_error))
        # ... (safety rule, see Part 5)
        registry = _injected_registry or _real_registry()
        function = registry.get(name)
        if function is None:
            return json.dumps(tool_envelope(error=f"unknown tool name: {name}"))

        result = function(**validated)
        # ...
        return json.dumps(result)
    except Exception as exc:
        return json.dumps(tool_envelope(error=f"tool execution failed: {exc}"))
```

tests/run_tests.py is a plain Python test runner using assert. It prints PASS or FAIL for each test and an X/Y summary. The tests call the real tool functions from transit_server.py, and only the database is replaced with a small in-memory data source, so they run offline without MySQL, a network connection or an LLM. I confirmed this by stopping the MySQL Docker container before running them.

Test	What it checks
test_valid_search	search_incidents with a valid query returns ok true
test_valid_get	get_incident with a valid code returns ok true
test_valid_aggregate	incident_count_by_route returns ok true
test_invalid_unknown_category	category "unknown-category" is rejected
test_invalid_incident_code	incident_code "abc" is rejected
test_invalid_minimum	min_incidents -5 is rejected
test_unknown_tool	an unknown tool name returns ok false
test_safety_limit_block (Part 5)	the safety rule blocks limit > 50
test_agent_stops_at_max_steps (Part 5)	run_agent with MockModel stops at max_steps

Code - tests/run_tests.py, one invalid-input test

```
def test_invalid_incident_code():
    result = payload(execute_tool("get_incident", {"incident_code": "abc"}))
    assert result == {
        "ok": False,
        "data": None,
        "error": "incident_code must match INC-#####",
    }
```

```
Kazi Samina Maraj Mumu
data260-4098-mysql
PASS test_valid_search
PASS test_valid_get
PASS test_valid_aggregate
PASS test_invalid_unknown_category
PASS test_invalid_incident_code
PASS test_invalid_minimum
PASS test_unknown_tool
PASS test_safety_limit_block
PASS test_agent_stops_at_max_steps
9/9 passed
data260-4098-mysql
((.venv) ) (base) samina@Kazis-MacBook-Pro-2 data260-4098 %
```

Figure 43. python tests/run_tests.py with the MySQL container stopped (docker stop / docker start around the run): PASS for every test and 9/9 passed.

Part 5 - Agent Loop, Safety Rule and Reflection

I. Safety Rule

I added one domain-specific safety rule inside `execute_tool`: `search_incidents` cannot request more than 50 records in one call. My database has 5,000 incidents with submitter emails, so this rule limits how much data the agent can pull out in a single call. It does not stop many repeated calls on its own; the `max_steps` limit in the agent loop puts a cap on that, and a rate limit would be the next step for a real system. When the rule is violated, `execute_tool` returns an error envelope instead of raising an exception. The rule is checked before the tool runs. I ran `code/mcp/safety_demo.py` to show one allowed and one blocked call (Figure 44).

Code - `code/mcp/execute_tool.py`, safety rule

```
# Safety rule: prevent bulk extraction through the single tool entry
# point while leaving the domain tool's normal limit of 50 unchanged.
if name == "search_incidents" and validated["limit"] > 50:
    return json.dumps(
        tool_envelope(
            error="safety policy: search limit cannot exceed 50"
        )
    )
```

```
Kazi Samina Maraj Mumu
allowed {"ok": true, "data": [{"id": 3, "incident_code": "INC-000003", "incident_title": "Transit D
blocked {"ok": false, "data": null, "error": "safety policy: search limit cannot exceed 50"}
((.venv) ) (base) samina@Kazis-MacBook-Pro-2 data260-4098 %
```

Figure 44. `code/mcp/safety_demo.py`: the allowed call (limit 10) returns `ok true` (output shortened to the first record), and the blocked call (limit 51) returns `ok false` with the safety policy error.

II. Agent Loop

`run_agent(user_input)` in `code/mcp/agent.py` uses the local `qwen3:8b` model through Ollama's `/api/chat` endpoint and the three domain tools through `execute_tool`. It has a turn counter and a `max_steps` limit. Each step is logged as one JSON line in `reports/hw05/raw/agent_runs.jsonl` with the `run_id`, `step`, `tool`, `input`, `result` and, on the last line, the `stop_reason` (completed, `safety_block` or `max_steps`).

In my first version, `qwen3` sometimes asked for several tools in one reply. The agent ran all of them as one step and then stopped because the tool-call count reached `max_steps`, so the table showed 1 step but a `max_steps` stop. I changed the loop so only one tool call runs per step and any extra calls are logged as `ignored_tool_calls`. Now the steps count 1, 2, 3 correctly.

Code - `code/mcp/agent.py`, `run_agent` loop (one tool call per step, turn counter and stop reasons)

```
for step in range(1, max_steps + 1):
    response = selected_model.chat(messages, TOOL_DEFINITIONS)
    calls = _extract_tool_calls(response)
    if not calls:
        # ... (no tool call -> stop_reason 'completed')
        call = calls[0]
        tool, inputs = _call_parts(call)
        result_text = execute_tool(tool, inputs)
        result = json.loads(result_text)
        tool_call_count += 1
    # ... (build log_entry, check safety_block)
    if safety_block:
        log_entry["stop_reason"] = "safety_block"
    elif step == max_steps:
        log_entry["stop_reason"] = "max_steps"
    _write_log(log_entry, selected_log_path)
```

III. Additional Tests

I added two tests to the Part 4 suite: `test_safety_limit_block` checks that `execute_tool` blocks a limit above 50, and `test_agent_stops_at_max_steps` uses a `MockModel` that always asks for a tool, and checks that `run_agent` stops at `max_steps`. Both run offline without an API key, a live model, an external API or the database. The tests also write to a temporary log file, so they never touch the real `agent_runs.jsonl`.

Code - `tests/run_tests.py`, `MockModel` test for `max_steps`

```
def test_agent_stops_at_max_steps():
    with tempfile.TemporaryDirectory() as temp_dir:
        result = run_agent(
            "Keep using the route-count tool.",
            model=MockModel(),
            max_steps=2,
            log_path=Path(temp_dir) / "agent_runs.jsonl",
        )
        assert result["stop_reason"] == "max_steps"
        assert result["steps"] == 2
        assert result["tool_calls"] == 2
```

IV. Metrics

I ran four scenarios with the local `qwen3:8b` model:

Scenario	Step count	Tool-call count	Stop reason
normal_search	2	1	completed
normal_route_counts	2	1	completed
safety_block	1	1	safety_block
max_steps (max_steps = 3)	3	3	max_steps

```
Kazi Samina Maraj Mumu
scenario | step_count | tool_call_count | stop_reason
-----|-----|-----|-----
normal_search | 2 | 1 | completed
normal_route_counts | 2 | 1 | completed
safety_block | 1 | 1 | safety_block
max_steps | 3 | 3 | max_steps
[2026-10-02T21:15:30Z] END python code/mcp/run_scenarios.py
((.venv) ) (base) samina@Kazis-MacBook-Pro-2 data260-4098 %
```

Figure 45. Scenario results from my final logged run of `code/mcp/run_scenarios.py` (`RUN_LOG.txt`).

The same table is in `reports/hw05/METRICS.md`.

V. Reflection

For this reflection I picked the `max_steps` scenario from `agent_runs.jsonl` (run 6170a6dc). In this run I asked the agent to look up INC-000002, then INC-000003, then INC-000004, then INC-000005 one at a time and summarize them at the end. The run used `qwen3:8b` through Ollama with `max_steps` set to 3.

At step 1, the harness sent my prompt and the three tool descriptions to the model. Instead of asking for one incident, the model asked for all four lookups at once. My harness only runs one tool call per step, so it ran `get_incident` for INC-000002 through `execute_tool` and recorded the other three as `ignored_tool_calls`. One of the ignored calls was even malformed: the INC-000005 request had a broken key ("`5name`") instead of a proper tool name. The INC-000002 result came back as `ok: true` in the `{ok, data, error}` envelope and was sent back to the model.

At step 2 and step 3 the model asked for INC-000003 and INC-000004, and both returned `ok: true`. In one of these steps it again asked for INC-000005 at the same time, and that extra call was also ignored. Nothing was blocked by the safety rule and there were no errors.

After step 3 the turn counter reached `max_steps`, so the harness stopped and logged `stop_reason` as `max_steps`. The model never got to look up INC-000005 or write its summary. So this run did not stop because the task was finished or because of the safety rule. It stopped because of the step ceiling.

This run showed me two things. The step limit stops a model that keeps asking for more work, and running only one tool call per step keeps the log easy to follow, even when the model sends several calls at once or a broken one.

Reproducibility and Objective Verification

I wrote `code/verify_hw05.py` and a Makefile target, `make verify-hw05`. It runs a smoke test on the tagged commit and writes `reports/hw05/verification.json` with the homework number, SID4, commit hash, model, SEED and `VERIFY_SEED`, the checks and a pass/fail for each. It does not change any application code.

Check	What it tests
FastAPI health on port 8498	GET /health returns 200 with <code>record_count</code>
meals MCP stdio tool call	<code>meals_server.py</code> starts and answers <code>search_meals_by_name</code>
transit MCP stdio tool call	<code>transit_server.py</code> starts and answers <code>incident_count_by_route</code>
Offline tests	<code>tests/run_tests.py</code> passes (9/9)
Required HW5 report files	<code>RUN_LOG</code> , <code>METRICS</code> , <code>AI_USE</code> , <code>REFLECTION</code> , raw data and report exist
Git commit hash	Records the commit that was verified

Commands I used to run the project:

```
source .venv/bin/activate
docker start data260-4098-mysql
PYTHONPATH=. python -m uvicorn src.api:app --port 8498
cd frontend && npm run dev
python code/run_fault_injection.py
python tests/run_tests.py
python code/mcp/run_scenarios.py
make verify-hw05
```

Evidence Files in `reports/hw05/`

- RUN_LOG.txt - real console output with timestamps for the runs in this report
- raw/fault_injection.csv - all 150 fault-injection call records
- raw/inspector/ - MCP Inspector outputs for all tools
- raw/agent_runs.jsonl - the agent run log from Part 5
- METRICS.md, CONTRACTS.md, AI_USE.md, REFLECTION.md, verification.json, report.pdf

AI Use Disclosure

1. What did I use an AI assistant for, and what did I do myself?

I used ChatGPT to help write the starting code for each part: the Route table and migration, the Redux slice, the two MCP servers, the retry wrapper, execute_tool, the agent loop and the verification script. I used Claude to help me plan my prompts, check ChatGPT's code before I ran it, understand error messages, and help draft my write-ups and this report from my results. I ran everything myself on my Mac. I backed up the database before the migration, ran the migration and checked that all 5,000 incidents were kept, tested every endpoint in Postman, tested the MCP tools in MCP Inspector, ran the fault-injection experiment and the Ollama scenarios, took the screenshots, and committed and pushed my work with Git.

2. What was one unsuitable AI result?

The first version of the Part 4 test runner passed 7/7, but it was not testing my real tools. The test file defined its own fake_search_incidents, fake_get_incident and fake_incident_count_by_route functions and passed those to execute_tool. So the invalid-input tests were checking the fake functions' validation, not the validation in transit_server.py.

3. How did I detect the problem?

After the tests passed, I checked the imports in tests/run_tests.py with grep. It showed the fake tool functions and no import from transit_server. That meant the tests would still pass even if my real tools had a validation bug.

4. What did I change, and why does it work now?

I asked for the tests to call the real tool functions from transit_server.py and to replace only the database with an in-memory data source. Now the unknown category, the malformed incident code and the negative min_incidents cases are rejected by my real code. I confirmed the tests still pass (7/7, and 9/9 after Part 5) with the MySQL container stopped, so they run offline.

Repository Link and Collaborator Access

Repository: <https://github.com/SaminaMaraj/data260-4098> (tag hw5). I opened the link and confirmed it works. I invited both required collaborators, Sbrikitha and supriyaselvanganesan, to the repository.

Conclusion

In this homework I added a Route entity with a real foreign key to my transit incident app, migrated 5,000 existing incidents without losing any, and moved the React client to Redux

Toolkit. I built two MCP servers, added timeouts and bounded retries, and measured them with seeded fault injection. Then I put the three domain tools behind one safe entry point with offline tests, added a safety rule, and connected everything to a local Ollama agent with a step limit. The most useful lesson for me was that passing tests and working code are not the same thing: several bugs only showed up when I checked what the code actually did.